Dashboard queries — actual SQL

Banknote & Coinzy · every tab's working query (events already expanded). Placeholders: `{PROJECT}` `{DATASET}` `{{start_date}}` `{{end_date}}`. `[[AND …]]` is added only when country/platform is filtered.

> Live order: summary table if refreshed → this SQL. Home / Product Analytics do not query BigQuery.

- **DAU events:** session_start, App_open, first_open
- **User id:** real GA4 user_id → param user_id (skip anonymous) → user_pseudo_id
- **Event names:** _android / _ios suffix stripped before matching

Compare Apps · Health report

> One query per app, then tagged. This is also the source for most MVP rate charts when the summary exists.

Banknote (shared raw) — product_daily_signals

`sql/dashboard/raw/16_product_daily_signals.sql`

```
-- ============================================================================
-- Raw-events daily product signals (one app / one dataset)
-- Used for Compare: run once per product, then union with product label.
-- dau = app_open_dau (session_start / App_open / first_open).
-- notification_dau and any_event_dau are supporting series, not mixed into dau.
-- ============================================================================

WITH base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', {{start_date}})
                          AND FORMAT_DATE('%Y%m%d', {{end_date}})
    AND _TABLE_SUFFIX NOT LIKE 'intraday_%'
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
)

SELECT
  event_date,
  COUNT(DISTINCT CASE WHEN event_name_base IN ('session_start', 'App_open', 'first_open') THEN resolved_user_id END) AS
dau,
  COUNT(DISTINCT CASE WHEN event_name_base IN ('session_start', 'App_open', 'first_open') THEN resolved_user_id END) AS
app_open_dau,
  COUNT(DISTINCT CASE WHEN event_name_base IN ('notification_display', 'notification_receive',
'notification_foreground', 'notification_open', 'notification_opened', 'notification_dismiss', 'notification_interact')
THEN resolved_user_id END) AS notification_dau,
  COUNT(DISTINCT resolved_user_id) AS any_event_dau,
  COUNT(DISTINCT CASE
    WHEN event_name_base = 'first_open'
    THEN resolved_user_id END) AS installs,
  COUNTIF(event_name_base IN (
    'identification_done_success', 'Identification_done_success'
  )) AS success_scans,
  COUNTIF(event_name_base IN (
    'identification_done_failure', 'Identification_done_failure',
    'Identification_failed', 'Identification_unsuccessful'
  )) AS failure_scans,
  SAFE_DIVIDE(
    COUNTIF(event_name_base IN ('identification_done_success', 'Identification_done_success')),
    COUNTIF(event_name_base IN (
      'identification_done_success', 'Identification_done_success',
      'identification_done_failure', 'Identification_done_failure',
      'Identification_failed', 'Identification_unsuccessful'
    ))
  ) AS identification_success_rate,
```

```
  SAFE_DIVIDE(
    COUNTIF(event_name_base IN ('identification_done_success', 'Identification_done_success')),
    COUNT(DISTINCT CASE WHEN event_name_base IN ('session_start', 'App_open', 'first_open') THEN resolved_user_id END)
  ) AS scans_per_dau,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Identified_limit_reached', 'identified_limit_reached',
      'identiifcation_limit_exceeded', 'identification_limit_exceeded',
      'scan_quota_exhausted', 'limit_exceeded',
      'free_scan_limit_exceeded', 'free_scan_blocked',
      'free_scan_success_quota_exhausted', 'free_scan_fail_quota_exhausted',
      'Identification_unsuccessful_limit_reached'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'identification_done_success', 'Identification_done_success',
      'identification_done_failure', 'Identification_done_failure',
      'Identification_failed', 'Identification_unsuccessful',
      'Identification_done'
    ) THEN resolved_user_id END)
  ) AS free_quota_hit_rate,
  SAFE_DIVIDE(
    COUNTIF(event_name_base IN (
      'Subs_confirm', 'subs_confirm', 'subs_confirm_discount',
      'paid_purchase', 'trial_purchase'
    )),
    COUNTIF(event_name_base IN (
      'Subs_page', 'Subs_page_discount', 'Subscription_screen',
      'Subs_page_onboarding', 'subscription_shown'
    ))
  ) AS paywall_to_confirm_rate,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'identification_done_success', 'Identification_done_success'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Identify_bottom_nav', 'Identify_home'
    ) THEN resolved_user_id END)
  ) AS open_to_success_rate,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Collection_screen', 'Global_catalogue_screen',
      'collection_bottom_nav', 'private_collection_bottom_nav'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN ('session_start', 'App_open', 'first_open') THEN resolved_user_id END)
  ) AS catalogue_open_rate,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'marketplace_screen', 'Marketplace_bottom_nav', 'marketplace_bottom_nav',
      'market_item_expolre'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN ('session_start', 'App_open', 'first_open') THEN resolved_user_id END)
  ) AS marketplace_engagement_rate,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'Subs_confirm', 'subs_confirm', 'subs_confirm_discount',
    'paid_purchase', 'trial_purchase'
  ) THEN resolved_user_id END) AS paying_users,
  COUNTIF(event_name_base IN (
    'Subs_page', 'Subs_page_discount', 'Subscription_screen',
    'Subs_page_onboarding', 'subscription_shown'
  )) AS paywall_impressions,
  COUNTIF(event_name_base IN (
    'Subs_confirm', 'subs_confirm', 'subs_confirm_discount',
    'paid_purchase', 'trial_purchase'
  )) AS purchase_confirms
FROM base
GROUP BY event_date
ORDER BY event_date;
```

Coinzy override — product_daily_signals

`sql/dashboard/product/coinzy/16_product_daily_signals.sql`

```
-- ============================================================================
-- Coinzy daily product signals (raw events) — Compare + MVP rollup
-- Verified from CoinzyAndroid: Collection_screen / collection_bottom_nav,
-- marketplace_screen / marketplace_bottom_nav, Subs_page / subs_confirm.
```

```
-- Cheap identity: GA4 user_id (skip placeholders) then user_pseudo_id — no event_params UNNEST.
-- ============================================================================

WITH base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', {{start_date}})
                          AND FORMAT_DATE('%Y%m%d', {{end_date}})
    AND _TABLE_SUFFIX NOT LIKE 'intraday_%'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
)

SELECT
  event_date,
  COUNT(DISTINCT CASE WHEN event_name_base IN ('session_start', 'App_open', 'first_open') THEN resolved_user_id END) AS
dau,
  COUNT(DISTINCT CASE WHEN event_name_base IN ('session_start', 'App_open', 'first_open') THEN resolved_user_id END) AS
app_open_dau,
  COUNT(DISTINCT CASE WHEN event_name_base IN ('notification_display', 'notification_receive',
'notification_foreground', 'notification_open', 'notification_opened', 'notification_dismiss', 'notification_interact')
THEN resolved_user_id END) AS notification_dau,
  COUNT(DISTINCT resolved_user_id) AS any_event_dau,
  COUNT(DISTINCT CASE
    WHEN event_name_base = 'first_open'
    THEN resolved_user_id END) AS installs,
  COUNTIF(event_name_base IN (
    'identification_done_success', 'Identification_done_success'
  )) AS success_scans,
  COUNTIF(event_name_base IN (
    'identification_done_failure', 'Identification_done_failure',
    'Identification_failed', 'Identification_unsuccessful'
  )) AS failure_scans,
  SAFE_DIVIDE(
    COUNTIF(event_name_base IN ('identification_done_success', 'Identification_done_success')),
    COUNTIF(event_name_base IN (
      'identification_done_success', 'Identification_done_success',
      'identification_done_failure', 'Identification_done_failure',
      'Identification_failed', 'Identification_unsuccessful'
    ))
  ) AS identification_success_rate,
  SAFE_DIVIDE(
    COUNTIF(event_name_base IN ('identification_done_success', 'Identification_done_success')),
    COUNT(DISTINCT CASE WHEN event_name_base IN ('session_start', 'App_open', 'first_open') THEN resolved_user_id END)
  ) AS scans_per_dau,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Identified_limit_reached', 'identified_limit_reached',
      'scan_quota_exhausted', 'limit_exceeded',
      'free_scan_limit_exceeded', 'free_scan_blocked',
      'free_scan_success_quota_exhausted', 'free_scan_fail_quota_exhausted',
      'Identification_unsuccessful_limit_reached',
      'identiifcation_limit_exceeded'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'identification_done_success', 'Identification_done_success',
      'identification_done_failure', 'Identification_done_failure',
      'Identification_failed', 'Identification_unsuccessful',
      'Identification_done'
    ) THEN resolved_user_id END)
  ) AS free_quota_hit_rate,
  SAFE_DIVIDE(
    COUNTIF(event_name_base IN (
      'subs_confirm', 'subs_confirm_discount', 'Subs_confirm',
      'paid_purchase', 'trial_purchase'
    )),
    COUNTIF(event_name_base IN (
      'Subs_page', 'Subs_page_discount', 'Subscription_screen',
      'Subs_page_onboarding', 'subscription_shown'
    ))
  ) AS paywall_to_confirm_rate,
  SAFE_DIVIDE(
```

```
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'identification_done_success', 'Identification_done_success'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Identify_bottom_nav', 'Identify_home'
    ) THEN resolved_user_id END)
  ) AS open_to_success_rate,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Collection_screen', 'Global_catalogue_screen', 'collection_bottom_nav'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN ('session_start', 'App_open', 'first_open') THEN resolved_user_id END)
  ) AS catalogue_open_rate,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'marketplace_screen', 'marketplace_bottom_nav', 'Marketplace_bottom_nav',
      'market_item_expolre'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN ('session_start', 'App_open', 'first_open') THEN resolved_user_id END)
  ) AS marketplace_engagement_rate,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'subs_confirm', 'subs_confirm_discount', 'Subs_confirm',
    'paid_purchase', 'trial_purchase'
  ) THEN resolved_user_id END) AS paying_users,
  COUNTIF(event_name_base IN (
    'Subs_page', 'Subs_page_discount', 'Subscription_screen',
    'Subs_page_onboarding', 'subscription_shown'
  )) AS paywall_impressions,
  COUNTIF(event_name_base IN (
    'subs_confirm', 'subs_confirm_discount', 'Subs_confirm',
    'paid_purchase', 'trial_purchase'
  )) AS purchase_confirms
FROM base
GROUP BY event_date
ORDER BY event_date;
```

1. DAU (opened app)

> MVP tab + Explorer Daily Active Users. Same events for both apps.

Query

`sql/dashboard/product/01_dau.sql · sql/dashboard/raw/01_dau.sql`

```
-- ================================================================================
-- MVP #1 — DAU from events_* (opened the app / session start = app_open_dau)
-- Does not count notification_display. Same definition for Banknote and Coinzy.
-- ================================================================================

SELECT
  PARSE_DATE('%Y%m%d', event_date) AS event_date,
  COUNT(DISTINCT COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null',
'undefined', '(not set)', '(anonymous)'), NULL, TRIM(user_id)), (SELECT TRIM(ep.value.string_value)
        FROM UNNEST(event_params) ep
        WHERE ep.key = 'user_id'
          AND ep.value.string_value IS NOT NULL
          AND TRIM(ep.value.string_value) != ''
          AND LOWER(TRIM(ep.value.string_value)) NOT IN ('anonymous', 'null', 'undefined', '(not set)', '(anonymous)')
        LIMIT 1), user_pseudo_id)) AS dau
FROM `{PROJECT}.{DATASET}.events_*`
WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', {{start_date}})
                        AND FORMAT_DATE('%Y%m%d', {{end_date}})
  AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
  AND REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')
  [[AND event_country = {{country}}]]
  [[AND event_platform = {{platform}}]]
GROUP BY event_date
ORDER BY event_date;
```

2. Install → first scan

> Never uses product_daily_signals. Join is user_pseudo_id only, same calendar day.

Banknote

sql/dashboard/product/banknote/02_time_to_first_scan.sql

```
-- ============================================================================
-- Banknote MVP #2 — Same-day first ID (device join)
-- See sql/dashboard/raw/02_time_to_first_scan.sql
-- ============================================================================

WITH bounds AS (
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    TIMESTAMP_MICROS(event_timestamp) AS event_ts,
    user_pseudo_id AS device_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND user_pseudo_id IS NOT NULL
    AND TRIM(user_pseudo_id) != ''
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
),
installs AS (
  SELECT
    device_id,
    event_date AS cohort_date,
    MIN(event_ts) AS first_at
  FROM base
  WHERE event_name_base = 'first_open'
     OR STARTS_WITH(event_name_base, 'first_open')
  GROUP BY device_id, event_date
),
same_day_success AS (
  SELECT
    device_id,
    event_date AS success_date,
    MIN(event_ts) AS success_at
  FROM base
  WHERE event_name_base IN ('identification_done_success', 'Identification_done_success')
  GROUP BY device_id, event_date
)
SELECT
  i.cohort_date AS event_date,
  COUNT(*) AS cohort_users,
  COUNTIF(s.device_id IS NOT NULL) AS users_scanned_day0,
  SAFE_DIVIDE(COUNTIF(s.device_id IS NOT NULL), COUNT(*)) AS day0_first_scan_rate,
  APPROX_QUANTILES(
    IF(s.success_at IS NULL, NULL, TIMESTAMP_DIFF(s.success_at, i.first_at, SECOND)),
    100
  )[OFFSET(50)] AS median_seconds_to_first_scan
FROM installs i
LEFT JOIN same_day_success s
  ON i.device_id = s.device_id
 AND i.cohort_date = s.success_date
GROUP BY i.cohort_date
ORDER BY i.cohort_date;
```

Coinzy

sql/dashboard/product/coinzy/02_time_to_first_scan.sql

```
-- ============================================================================
```

```
-- Coinzy MVP #2 — Same-day first ID (device join)
-- See sql/dashboard/raw/02_time_to_first_scan.sql
-- ============================================================================

WITH bounds AS (
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    TIMESTAMP_MICROS(event_timestamp) AS event_ts,
    user_pseudo_id AS device_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND user_pseudo_id IS NOT NULL
    AND TRIM(user_pseudo_id) != ''
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
),
installs AS (
  SELECT
    device_id,
    event_date AS cohort_date,
    MIN(event_ts) AS first_at
  FROM base
  WHERE event_name_base = 'first_open'
     OR STARTS_WITH(event_name_base, 'first_open')
  GROUP BY device_id, event_date
),
same_day_success AS (
  SELECT
    device_id,
    event_date AS success_date,
    MIN(event_ts) AS success_at
  FROM base
  WHERE event_name_base IN ('identification_done_success', 'Identification_done_success')
  GROUP BY device_id, event_date
)
SELECT
  i.cohort_date AS event_date,
  COUNT(*) AS cohort_users,
  COUNTIF(s.device_id IS NOT NULL) AS users_scanned_day0,
  SAFE_DIVIDE(COUNTIF(s.device_id IS NOT NULL), COUNT(*)) AS day0_first_scan_rate,
  APPROX_QUANTILES(
    IF(s.success_at IS NULL, NULL, TIMESTAMP_DIFF(s.success_at, i.first_at, SECOND)),
    100
  )[OFFSET(50)] AS median_seconds_to_first_scan
FROM installs i
LEFT JOIN same_day_success s
  ON i.device_id = s.device_id
 AND i.cohort_date = s.success_date
GROUP BY i.cohort_date
ORDER BY i.cohort_date;
```

3. Identify success

> Rate = success events ÷ (success + failure events).

Coinzy (raw events_*)

`sql/dashboard/product/coinzy/03_identify_success_rate.sql`

```
-- ============================================================================
-- Coinzy MVP #3 — Identification success rate (raw events)
-- Success / failure from CoinAnalysisScreen.kt
-- ============================================================================

WITH bounds AS (
```

```
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
)
SELECT
  event_date,
  COUNTIF(event_name_base IN ('identification_done_success', 'Identification_done_success')) AS success_events,
  COUNTIF(event_name_base IN (
    'identification_done_failure', 'Identification_done_failure',
    'Identification_failed', 'Identification_unsuccessful'
  )) AS failure_events,
  SAFE_DIVIDE(
    COUNTIF(event_name_base IN ('identification_done_success', 'Identification_done_success')),
    COUNTIF(event_name_base IN (
      'identification_done_success', 'Identification_done_success',
      'identification_done_failure', 'Identification_done_failure',
      'Identification_failed', 'Identification_unsuccessful'
    ))
  ) AS identification_success_rate
FROM base
GROUP BY event_date
ORDER BY event_date;
```

Banknote — view the product SQL reads (event matching)

`sql/08_v_identify_metrics.sql (dashboard/product/03_identify_success_rate.sql selects from this view)`

```
-- ============================================================================
-- v_identify_metrics
-- Daily Identify funnel + quality + quota pressure for Banknote.
-- Metrics: #6 funnel, #7 success, #8 no-match, #9 quota, #19 camera permission
-- (Time-to-first-scan is v_time_to_first_scan — separate grain.)
-- Depends on: v_events_normalized
-- ============================================================================

CREATE OR REPLACE VIEW `{PROJECT}.{DATASET}.v_identify_metrics` AS

WITH events AS (
  SELECT
    event_date,
    platform,
    country,
    app_version,
    resolved_user_id,
    event_name_base,
    option_number
  FROM `{PROJECT}.{DATASET}.v_events_normalized`
  WHERE resolved_user_id IS NOT NULL
),

daily_funnel AS (
  SELECT
    event_date,
    platform,
    country,
    app_version,

    COUNT(DISTINCT CASE
      WHEN event_name_base IN (
        'Identify_bottom_nav', 'Identify_home'
      ) THEN resolved_user_id END) AS users_identify_open,

    COUNT(DISTINCT CASE
      WHEN event_name_base IN (
        'camera_permission_granted', 'Camera_permission_granted',
        'permission_camera_granted'
```

```
    ) THEN resolved_user_id END) AS users_camera_granted,

  COUNT(DISTINCT CASE
    WHEN event_name_base IN (
      'camera_permission_denied', 'Camera_permission_denied',
      'permission_camera_denied', 'camer_permission_denied'
    ) THEN resolved_user_id END) AS users_camera_denied,

  COUNT(DISTINCT CASE
    WHEN event_name_base IN (
      'photo_clicked_1', 'photo_clicked_2', 'Photo_clicked',
      'photo_captured', 'Photo_captured', 'photo_taken', 'Image_captured'
    ) THEN resolved_user_id END) AS users_photo_captured,

  COUNT(DISTINCT CASE
    WHEN event_name_base IN (
      'photo_submit_button', 'photos_submitted',
      'identification_submit', 'Identification_submit', 'Identification_done'
    ) THEN resolved_user_id END) AS users_submit,

  COUNT(DISTINCT CASE
    WHEN event_name_base IN (
      'identification_done_success', 'Identification_done_success'
    ) THEN resolved_user_id END) AS users_success,

  COUNT(DISTINCT CASE
    WHEN event_name_base IN (
      'identification_done_failure', 'Identification_done_failure',
      'Identification_failed', 'Identification_unsuccessful'
    ) THEN resolved_user_id END) AS users_failure,

  COUNTIF(event_name_base IN (
    'identification_done_success', 'Identification_done_success'
  )) AS success_events,

  COUNTIF(event_name_base IN (
    'identification_done_failure', 'Identification_done_failure',
    'Identification_failed', 'Identification_unsuccessful'
  )) AS failure_events,

  COUNTIF(event_name_base IN (
    'identification_report_no_match', 'Identification_report_no_match',
    'no_match', 'Identification_no_match'
  )) AS no_match_events,

  COUNTIF(event_name_base IN (
    'Identification_view_all', 'identification_view_all',
    'view_all_options'
  )) AS view_all_events,

  COUNTIF(
    event_name_base IN (
      'Identification_option_selected', 'identification_option_selected',
      'idetnification_option_chosen', 'identification_option_chosen'
    )
    OR option_number IS NOT NULL
  ) AS multi_option_events,

  COUNTIF(event_name_base IN (
    'Identified_limit_reached', 'identified_limit_reached',
    'identiifcation_limit_exceeded', 'identification_limit_exceeded',
    'scan_quota_exhausted', 'Scan_quota_exhausted', 'limit_exceeded', 'Limit_exceeded',
      'free_scan_limit_exceeded', 'free_scan_blocked',
      'free_scan_success_quota_exhausted', 'free_scan_fail_quota_exhausted',
      'Identification_unsuccessful_limit_reached'
    )) AS quota_hit_events,

  COUNT(DISTINCT CASE
    WHEN event_name_base IN (
      'Identified_limit_reached', 'identified_limit_reached',
      'identiifcation_limit_exceeded', 'identification_limit_exceeded',
      'scan_quota_exhausted', 'Scan_quota_exhausted',
      'limit_exceeded', 'Limit_exceeded',
      'free_scan_limit_exceeded', 'free_scan_blocked',
      'free_scan_success_quota_exhausted', 'free_scan_fail_quota_exhausted',
      'Identification_unsuccessful_limit_reached'
    ) THEN resolved_user_id END) AS users_hit_quota,
```

```
    COUNT(DISTINCT CASE
      WHEN event_name_base IN (
        'identification_done_success', 'Identification_done_success',
        'identification_done_failure', 'Identification_done_failure',
        'Identification_failed', 'Identification_unsuccessful',
        'Identification_done'
      ) THEN resolved_user_id END) AS users_attempted_scan

  FROM events
  GROUP BY event_date, platform, country, app_version
)

SELECT
  *,
  SAFE_DIVIDE(success_events, success_events + failure_events)
    AS identification_success_rate,
  SAFE_DIVIDE(no_match_events, success_events + failure_events + no_match_events)
    AS no_match_rate,
  SAFE_DIVIDE(view_all_events + multi_option_events, NULLIF(success_events, 0))
    AS distrust_signal_rate,
  SAFE_DIVIDE(users_hit_quota, users_attempted_scan)
    AS free_quota_hit_rate,
  SAFE_DIVIDE(users_camera_granted, users_camera_granted + users_camera_denied)
    AS camera_permission_grant_rate,
  SAFE_DIVIDE(users_success, users_identify_open) AS identify_open_to_success_rate,
  SAFE_DIVIDE(users_photo_captured, users_identify_open) AS identify_open_to_photo_rate,
  SAFE_DIVIDE(users_success, users_submit) AS submit_to_success_rate
FROM daily_funnel;
```

4. Quota hit

> Users who hit scan quota ÷ users who attempted a scan. Not collection limit.

Banknote

`sql/dashboard/product/banknote/04_quota_hit_rate.sql`

```
-- ================================================================================
-- Banknote MVP #4 — Quota hit rate (raw events)
-- App fires identiifcation_limit_exceeded (typo) — see funnel-registry.js
-- ================================================================================

WITH bounds AS (
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(
      user_id,
      (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
      user_pseudo_id
    ) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
)
SELECT
  event_date,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'Identified_limit_reached', 'identified_limit_reached',
    'identiifcation_limit_exceeded', 'identification_limit_exceeded',
    'scan_quota_exhausted', 'Scan_quota_exhausted', 'limit_exceeded', 'Limit_exceeded'
  ) THEN resolved_user_id END) AS users_hit_quota,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'identification_done_success', 'Identification_done_success',
```

```
    'identification_done_failure', 'Identification_done_failure',
    'Identification_done'
  ) THEN resolved_user_id END) AS users_attempted_scan,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Identified_limit_reached', 'identified_limit_reached',
      'identiifcation_limit_exceeded', 'identification_limit_exceeded',
      'scan_quota_exhausted', 'Scan_quota_exhausted', 'limit_exceeded', 'Limit_exceeded'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'identification_done_success', 'Identification_done_success',
      'identification_done_failure', 'Identification_done_failure',
      'Identification_done'
    ) THEN resolved_user_id END)
  ) AS free_quota_hit_rate
FROM base
GROUP BY event_date
ORDER BY event_date;
```

Coinzy

sql/dashboard/product/coinzy/04_quota_hit_rate.sql

```
-- ================================================================================
-- Coinzy MVP #4 — Quota hit rate (raw events)
-- Identified_limit_reached + free_scan_* from FreeScanLimitUtil.kt / ExperimentUtil.kt
-- ================================================================================

WITH bounds AS (
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(
      user_id,
      (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
      user_pseudo_id
    ) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
)
SELECT
  event_date,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'Identified_limit_reached', 'identified_limit_reached',
    'free_scan_limit_exceeded', 'free_scan_blocked',
    'free_scan_success_quota_exhausted', 'free_scan_fail_quota_exhausted',
    'Identification_unsuccessful_limit_reached',
    'identiifcation_limit_exceeded',
    'scan_quota_exhausted', 'limit_exceeded'
  ) THEN resolved_user_id END) AS users_hit_quota,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'identification_done_success', 'Identification_done_success',
    'identification_done_failure', 'Identification_done_failure',
    'Identification_failed', 'Identification_unsuccessful',
    'Identification_done'
  ) THEN resolved_user_id END) AS users_attempted_scan,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Identified_limit_reached', 'identified_limit_reached',
      'free_scan_limit_exceeded', 'free_scan_blocked',
      'free_scan_success_quota_exhausted', 'free_scan_fail_quota_exhausted',
      'Identification_unsuccessful_limit_reached',
      'identiifcation_limit_exceeded',
      'scan_quota_exhausted', 'limit_exceeded'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'identification_done_success', 'Identification_done_success',
      'identification_done_failure', 'Identification_done_failure',
```

```
      'Identification_failed', 'Identification_unsuccessful',
      'Identification_done'
    ) THEN resolved_user_id END)
  ) AS free_quota_hit_rate
FROM base
GROUP BY event_date
ORDER BY event_date;
```

5. Paywall → purchase

> Confirm events ÷ paywall events. Banknote confirm is Subs_confirm; Coinzy is subs_confirm / paid_purchase.

Coinzy (raw events_*)

`sql/dashboard/product/coinzy/05_paywall_conversion.sql`

```
-- ============================================================================
-- Coinzy MVP #5 — Paywall → purchase (raw events)
-- Confirm is lowercase subs_confirm (BillingViewModel.kt) — NOT Subs_confirm
-- ============================================================================

WITH bounds AS (
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(
      user_id,
      (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
      user_pseudo_id
    ) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
)
SELECT
  event_date,
  COUNTIF(event_name_base IN (
    'Subs_page', 'Subs_page_discount', 'Subscription_screen',
    'Subs_page_onboarding', 'subscription_shown'
  )) AS paywall_impressions,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'Subs_page', 'Subs_page_discount', 'Subscription_screen',
    'Subs_page_onboarding', 'subscription_shown'
  ) THEN resolved_user_id END) AS users_saw_paywall,
  COUNTIF(event_name_base IN (
    'subs_confirm', 'subs_confirm_discount', 'Subs_confirm',
    'paid_purchase', 'trial_purchase'
  )) AS purchase_confirms,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'subs_confirm', 'subs_confirm_discount', 'Subs_confirm',
    'paid_purchase', 'trial_purchase'
  ) THEN resolved_user_id END) AS paying_users,
  SAFE_DIVIDE(
    COUNTIF(event_name_base IN (
      'subs_confirm', 'subs_confirm_discount', 'Subs_confirm',
      'paid_purchase', 'trial_purchase'
    )),
    COUNTIF(event_name_base IN (
      'Subs_page', 'Subs_page_discount', 'Subscription_screen',
      'Subs_page_onboarding', 'subscription_shown'
    ))
  ) AS paywall_to_confirm_rate,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'subs_confirm', 'subs_confirm_discount', 'Subs_confirm',
```

```
      'paid_purchase', 'trial_purchase'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Subs_page', 'Subs_page_discount', 'Subscription_screen',
      'Subs_page_onboarding', 'subscription_shown'
    ) THEN resolved_user_id END)
  ) AS user_paywall_conversion_rate
FROM base
GROUP BY event_date
ORDER BY event_date;
```

Banknote — view the product SQL reads

sql/07_v_subscription_metrics.sql

```
-- ================================================================================
-- v_subscription_metrics
-- Daily subscription funnel and outcome metrics.
-- Depends on: v_events_normalized
-- ================================================================================

CREATE OR REPLACE VIEW `{PROJECT}.{DATASET}.v_subscription_metrics` AS

WITH subs_events AS (
  SELECT
    event_date,
    platform,
    country,
    app_version,
    resolved_user_id,
    event_name_base,
    pack_name,
    discounted_type,
    action,
    error
  FROM `{PROJECT}.{DATASET}.v_events_normalized`
  WHERE event_name_base IN (
      'Subs_page',
      'Subs_page_discount',
      'Subscription_screen',
      'Subs_page_onboarding',
      'Subs_pack',
      'subs_pack',
      'subs_pack_discount',
      'subs_button',
      'Subs_confirm',
      'subs_confirm',
      'subs_confirm_discount',
      'paid_purchase',
      'trial_purchase',
      'Subs_fail',
      'subs_fail',
      'Subs_cancel',
      'subs_cancel',
      'Subs_restore',
      'subs_native',
      'go_pro_button',
      'subs_discount_banner',
      'subscription_shown'
    )
    OR event_name_base IN ('subscription', 'life_time_access', 'subscription_management_opened')
),

daily AS (
  SELECT
    event_date,
    platform,
    country,
    app_version,

    -- Funnel top
    COUNT(DISTINCT CASE
      WHEN event_name_base IN (
        'Subs_page', 'Subs_page_discount', 'Subscription_screen',
        'Subs_page_onboarding', 'subscription_shown'
      )
```

```
      THEN resolved_user_id END) AS users_saw_paywall,

    COUNTIF(event_name_base IN (
      'Subs_page', 'Subs_page_discount', 'Subscription_screen',
      'Subs_page_onboarding', 'subscription_shown'
    )) AS paywall_impressions,
    COUNTIF(event_name_base = 'Subs_page')                          AS paywall_standard_impressions,
    COUNTIF(event_name_base = 'Subs_page_discount')                 AS paywall_discount_impressions,

    -- Pack selection (Banknote Subs_pack; Coinzy subs_pack)
    COUNTIF(event_name_base IN ('Subs_pack', 'subs_pack', 'subs_pack_discount')) AS pack_clicks,
    COUNT(DISTINCT CASE
      WHEN event_name_base IN ('Subs_pack', 'subs_pack', 'subs_pack_discount')
      THEN resolved_user_id END) AS users_clicked_pack,

    -- Outcomes (Banknote Subs_confirm; Coinzy subs_confirm)
    COUNTIF(event_name_base IN (
      'Subs_confirm', 'subs_confirm', 'subs_confirm_discount',
      'paid_purchase', 'trial_purchase'
    )) AS purchase_confirms,
    COUNTIF(event_name_base = 'subscription')                       AS direct_subscription_events,
    COUNTIF(event_name_base = 'life_time_access')                   AS lifetime_purchases,
    COUNTIF(event_name_base IN ('Subs_fail', 'subs_fail'))           AS purchase_failures,
    COUNTIF(event_name_base IN ('Subs_cancel', 'subs_cancel'))       AS purchase_cancels,
    COUNTIF(event_name_base = 'Subs_restore')                      AS restore_attempts,

    COUNT(DISTINCT CASE
      WHEN event_name_base IN (
        'Subs_confirm', 'subs_confirm', 'subs_confirm_discount', 'paid_purchase', 'trial_purchase'
      ) THEN resolved_user_id END) AS paying_users,

    -- Entry points
    COUNTIF(event_name_base = 'go_pro_button')                      AS go_pro_clicks,
    COUNTIF(event_name_base = 'subs_discount_banner')               AS discount_banner_clicks

  FROM subs_events
  GROUP BY event_date, platform, country, app_version
)

SELECT
  *,
  SAFE_DIVIDE(purchase_confirms, paywall_impressions)     AS paywall_to_confirm_rate,
  SAFE_DIVIDE(pack_clicks, paywall_impressions)           AS paywall_to_pack_click_rate,
  SAFE_DIVIDE(purchase_confirms, pack_clicks)             AS pack_click_to_confirm_rate,
  SAFE_DIVIDE(purchase_failures, purchase_failures + purchase_confirms) AS purchase_failure_rate
FROM daily;
```

6. D1 / D7 retention

> Cohort = first_open. Returned = any Firebase event on D+1 / D+7. Same SQL for MVP 6 and Explorer D1/D7 (merged).

Query

`sql/dashboard/raw/09_retention.sql`

```
-- ============================================================================
-- Raw-events D1 + D7 retention (no views required)
-- Cohort = first_open day; retained = any activity on cohort_date + N days.
-- Activity window extends end_date by 7 days so D7 cohorts can mature.
-- ============================================================================

WITH params AS (
  SELECT
    {{start_date}} AS start_date,
    {{end_date}} AS end_date,
    DATE_ADD({{end_date}}, INTERVAL 7 DAY) AS activity_end
),

user_events AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
```

```
    COALESCE(
      user_id,
      (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
      user_pseudo_id
    ) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, params
  WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', start_date)
                          AND FORMAT_DATE('%Y%m%d', activity_end)
    AND _TABLE_SUFFIX NOT LIKE 'intraday_%'
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
),

cohorts AS (
  SELECT
    resolved_user_id,
    MIN(event_date) AS cohort_date
  FROM user_events
  WHERE event_name_base IN ('first_open')
     OR REGEXP_CONTAINS(event_name_base, r'^first_open')
  GROUP BY resolved_user_id
),

activity AS (
  SELECT DISTINCT
    resolved_user_id,
    event_date AS activity_date
  FROM user_events
),

cohort_flags AS (
  SELECT
    c.cohort_date,
    c.resolved_user_id,
    MAX(IF(a.activity_date = DATE_ADD(c.cohort_date, INTERVAL 1 DAY), 1, 0)) AS returned_d1,
    MAX(IF(a.activity_date = DATE_ADD(c.cohort_date, INTERVAL 7 DAY), 1, 0)) AS returned_d7
  FROM cohorts c
  CROSS JOIN params p
  LEFT JOIN activity a
    ON c.resolved_user_id = a.resolved_user_id
   AND a.activity_date IN (
     DATE_ADD(c.cohort_date, INTERVAL 1 DAY),
     DATE_ADD(c.cohort_date, INTERVAL 7 DAY)
   )
  WHERE c.cohort_date BETWEEN p.start_date AND p.end_date
  GROUP BY c.cohort_date, c.resolved_user_id
)

SELECT
  cohort_date,
  COUNT(*) AS cohort_size,
  SUM(returned_d1) AS retained_d1,
  SUM(returned_d7) AS retained_d7,
  SAFE_DIVIDE(SUM(returned_d1), COUNT(*)) AS d1_retention_rate,
  SAFE_DIVIDE(SUM(returned_d7), COUNT(*)) AS d7_retention_rate
FROM cohort_flags
WHERE DATE_ADD(cohort_date, INTERVAL 1 DAY) <= CURRENT_DATE()
GROUP BY cohort_date
ORDER BY cohort_date;
```

7. Scans / user · 9. Catalogue · 10. Marketplace

> When signals miss: Coinzy uses dedicated raw SQL. Banknote reads v_engagement_metrics (included once).

Coinzy — scans / user

`sql/dashboard/product/coinzy/07_scans_per_user.sql`

```
-- ============================================================================
-- Coinzy MVP #7 — Scans per active user (raw events)
-- ============================================================================
```

```
WITH bounds AS (
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(
      user_id,
      (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
      user_pseudo_id
    ) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
)
SELECT
  event_date,
  COUNT(DISTINCT resolved_user_id) AS dau,
  COUNTIF(event_name_base IN (
    'identification_done_success', 'Identification_done_success'
  )) AS success_scans,
  SAFE_DIVIDE(
    COUNTIF(event_name_base IN (
      'identification_done_success', 'Identification_done_success'
    )),
    COUNT(DISTINCT resolved_user_id)
  ) AS scans_per_dau,
  SAFE_DIVIDE(
    COUNTIF(event_name_base IN (
      'identification_done_success', 'Identification_done_success'
    )),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'identification_done_success', 'Identification_done_success'
    ) THEN resolved_user_id END)
  ) AS scans_per_scanning_user
FROM base
GROUP BY event_date
ORDER BY event_date;
```

Coinzy — catalogue

`sql/dashboard/product/coinzy/09_catalogue_engagement.sql`

```
-- ================================================================================
-- Coinzy MVP #9 — Catalogue / collection engagement (raw events)
-- Opens: Collection_screen ∪ Global_catalogue_screen
-- Detail: Coin_details*  ·  Add: Added_to_collection* (+ space typo)
-- ================================================================================

WITH bounds AS (
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(
      user_id,
      (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
      user_pseudo_id
    ) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
),
flags AS (
```

```
  SELECT
    event_date,
    resolved_user_id,
    MAX(IF(event_name_base IN (
      'Collection_screen', 'Global_catalogue_screen', 'collection_bottom_nav'
    ), 1, 0)) AS opened,
    MAX(IF(event_name_base IN (
      'Coin_details', 'Coin_details_collection', 'Coin_details_global',
      'Coin_details_identification', 'identification_details_screen'
    ), 1, 0)) AS viewed_detail,
    MAX(IF(event_name_base IN (
      'Filter_button_private', 'Filter_button_global',
      'filter_field_selected_collection', 'filter_sub_collection',
      'Predefined_filter_private', 'Predefined_filter_global'
    ), 1, 0)) AS used_filter,
    MAX(IF(event_name_base IN (
      'identification_done_success', 'Identification_done_success'
    ), 1, 0)) AS had_success,
    MAX(IF(
      STARTS_WITH(event_name_base, 'Added_to_collection')
      OR event_name_base IN ('Added _to_collection_owned', 'add_to_wishlist'),
      1, 0
    )) AS added
  FROM base
  GROUP BY event_date, resolved_user_id
)
SELECT
  event_date,
  COUNT(*) AS dau,
  COUNTIF(opened = 1) AS users_opened_collection,
  COUNTIF(viewed_detail = 1) AS users_viewed_detail,
  COUNTIF(used_filter = 1) AS users_used_filter,
  COUNTIF(had_success = 1) AS users_with_success_id,
  COUNTIF(had_success = 1 AND added = 1) AS users_added_after_id,
  SAFE_DIVIDE(COUNTIF(opened = 1), COUNT(*)) AS catalogue_open_rate,
  SAFE_DIVIDE(COUNTIF(viewed_detail = 1), COUNT(*)) AS catalogue_detail_rate,
  SAFE_DIVIDE(
    COUNTIF(had_success = 1 AND added = 1),
    COUNTIF(had_success = 1)
  ) AS collection_add_rate_after_id
FROM flags
GROUP BY event_date
ORDER BY event_date;
```

Coinzy — marketplace (Feed is a separate column, not mixed into marketplace rate)

`sql/dashboard/product/coinzy/10_marketplace_engagement.sql`

```
-- ============================================================================
-- Coinzy MVP #10 — Marketplace + Feed engagement (raw events)
-- Screen: marketplace_screen  ·  Contact: market_contact / market_contact_button
-- NOT Marketplace_open / contact_seller
-- ============================================================================

WITH bounds AS (
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(
      user_id,
      (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
      user_pseudo_id
    ) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
),
flags AS (
  SELECT
```

```
    event_date,
    resolved_user_id,
    MAX(IF(event_name_base IN (
      'marketplace_screen', 'marketplace_bottom_nav', 'Marketplace_bottom_nav',
      'market_item_expolre'
    ), 1, 0)) AS marketplace,
    MAX(IF(event_name_base IN ('market_contact', 'market_contact_button'), 1, 0)) AS contacted,
    MAX(IF(event_name_base IN (
      'Feed_screen', 'feed_bottom_nav', 'feed_like', 'feed_comment', 'feed_add'
    ), 1, 0)) AS feed
  FROM base
  GROUP BY event_date, resolved_user_id
)
SELECT
  event_date,
  COUNT(*) AS dau,
  COUNTIF(marketplace = 1) AS users_marketplace,
  COUNTIF(contacted = 1) AS users_contacted_seller,
  COUNTIF(feed = 1) AS users_feed,
  SAFE_DIVIDE(COUNTIF(marketplace = 1), COUNT(*)) AS marketplace_engagement_rate,
  SAFE_DIVIDE(COUNTIF(contacted = 1), COUNTIF(marketplace = 1)) AS contact_seller_rate,
  SAFE_DIVIDE(COUNTIF(feed = 1), COUNT(*)) AS feed_engagement_rate
FROM flags
GROUP BY event_date
ORDER BY event_date;
```

Banknote — view for scans / catalogue / marketplace

`sql/10_v_engagement_metrics.sql`

```
-- ============================================================================
-- v_engagement_metrics
-- Daily engagement intensity + collection / marketplace / feed loops.
-- Metrics: #15 scans/user, #16 collection add after ID, #17 catalogue,
--          #18 marketplace & feed, #20 session length + feature mix
-- Depends on: v_events_normalized, v_daily_active_users
-- ============================================================================

CREATE OR REPLACE VIEW `{PROJECT}.{DATASET}.v_engagement_metrics` AS

WITH dau AS (
  SELECT
    event_date,
    platform,
    country,
    COUNT(DISTINCT resolved_user_id) AS dau,
    SUM(identifications_success) AS total_success_scans,
    SUM(identifications) AS total_identifications,
    COUNT(DISTINCT CASE WHEN identifications_success > 0 THEN resolved_user_id END)
      AS users_with_scan
  FROM `{PROJECT}.{DATASET}.v_daily_active_users`
  GROUP BY event_date, platform, country
),

user_day_flags AS (
  SELECT
    event_date,
    platform,
    country,
    resolved_user_id,
    MAX(IF(event_name_base IN (
      'identification_done_success', 'Identification_done_success'
    ), 1, 0)) AS had_success_id,
    MAX(IF(
      STARTS_WITH(event_name_base, 'Added_to_collection')
      OR event_name_base IN (
        'Added_to_collection', 'add_to_collection',
        'Collection_add', 'collection_item_added',
        'Added _to_collection_owned', 'add_to_wishlist'
      ), 1, 0)) AS added_to_collection,
    MAX(IF(event_name_base IN (
      'Collection_screen', 'Global_catalogue_screen',
      'collection_bottom_nav', 'private_collection_bottom_nav'
    ), 1, 0)) AS opened_collection,
    MAX(IF(event_name_base IN (
      'Collection_detail', 'collection_detail', 'banknote_detail',
```

```
      'coin_detail', 'Catalogue_detail', 'note_detail_view',
      'Coin_details', 'Coin_details_collection', 'Coin_details_global',
      'Coin_details_identification', 'identification_details_screen',
      'banknote_details_collection', 'banknote_details_global',
      'banknote_details_identification'
    ), 1, 0)) AS viewed_detail,
    MAX(IF(
      event_name_base IN (
        'Collection_filter', 'filter_applied', 'Filter_applied',
        'Filter_button_private', 'Filter_button_global',
        'filter_field_selected_collection', 'filter_sub_collection',
        'Predefined_filter_private', 'Predefined_filter_global'
      )
      OR filter_name IS NOT NULL,
      1, 0
    )) AS used_filter,
    MAX(IF(event_name_base IN (
      'marketplace_screen', 'Marketplace_bottom_nav', 'marketplace_bottom_nav',
      'market_item_expolre'
    ), 1, 0)) AS marketplace_engaged,
    MAX(IF(event_name_base IN (
      'market_contact', 'market_contact_button',
      'contact_seller', 'Contact_seller', 'Marketplace_contact'
    ), 1, 0)) AS contacted_seller,
    MAX(IF(event_name_base IN (
      'Feed_screen', 'feed_bottom_nav', 'feed_like', 'feed_comment', 'feed_add',
      'Feed_open', 'feed_open', 'Feed_post', 'Feed_like', 'post_like'
    ), 1, 0)) AS feed_engaged,
    MAX(IF(event_name_base IN (
      'Homescreen', 'home_bottom_nav', 'Home', 'Home_open', 'home_open'
    ), 1, 0)) AS used_home,
    MAX(IF(event_name_base IN (
      'Identify_bottom_nav', 'Identify_home'
    ), 1, 0)) AS used_identify,
    AVG(IF(session_length_seconds > 0, session_length_seconds, NULL)) AS avg_session_length
  FROM `{PROJECT}.{DATASET}.v_events_normalized`
  WHERE resolved_user_id IS NOT NULL
  GROUP BY event_date, platform, country, resolved_user_id
),

engagement AS (
  SELECT
    event_date,
    platform,
    country,
    COUNTIF(had_success_id = 1) AS users_with_success_id,
    COUNTIF(had_success_id = 1 AND added_to_collection = 1) AS users_added_after_id,
    COUNTIF(opened_collection = 1) AS users_opened_collection,
    COUNTIF(viewed_detail = 1) AS users_viewed_detail,
    COUNTIF(used_filter = 1) AS users_used_filter,
    COUNTIF(marketplace_engaged = 1) AS users_marketplace,
    COUNTIF(contacted_seller = 1) AS users_contacted_seller,
    COUNTIF(feed_engaged = 1) AS users_feed,
    COUNTIF(used_home = 1) AS users_home,
    COUNTIF(used_identify = 1) AS users_identify_tab,
    AVG(avg_session_length) AS avg_session_length_seconds
  FROM user_day_flags
  GROUP BY event_date, platform, country
)

SELECT
  d.event_date,
  d.platform,
  d.country,
  d.dau,
  d.total_success_scans,
  d.total_identifications,
  d.users_with_scan,

  -- #15 Scans per active user
  SAFE_DIVIDE(d.total_success_scans, d.dau) AS scans_per_dau,
  SAFE_DIVIDE(d.total_success_scans, d.users_with_scan) AS scans_per_scanning_user,

  -- #16 Collection add after identify
  e.users_with_success_id,
  e.users_added_after_id,
  SAFE_DIVIDE(e.users_added_after_id, e.users_with_success_id) AS collection_add_rate_after_id,
```

```
  -- #17 Collection / catalogue
  e.users_opened_collection,
  e.users_viewed_detail,
  e.users_used_filter,
  SAFE_DIVIDE(e.users_opened_collection, d.dau) AS collection_open_rate,

  -- #18 Marketplace & Feed
  e.users_marketplace,
  e.users_contacted_seller,
  e.users_feed,
  SAFE_DIVIDE(e.users_marketplace, d.dau) AS marketplace_engagement_rate,
  SAFE_DIVIDE(e.users_feed, d.dau) AS feed_engagement_rate,

  -- #20 Feature mix
  e.users_home,
  e.users_identify_tab,
  e.avg_session_length_seconds,
  SAFE_DIVIDE(e.users_home, d.dau) AS home_usage_rate,
  SAFE_DIVIDE(e.users_identify_tab, d.dau) AS identify_tab_usage_rate

FROM dau d
LEFT JOIN engagement e
  ON d.event_date = e.event_date
 AND d.platform = e.platform
 AND d.country = e.country;
```

8. Identify funnel (open → success rate chart)

> This tab is the daily rate. Step drop-off is the Identify funnel pages (next section).

Coinzy

`sql/dashboard/product/coinzy/08_identify_funnel_conversion.sql`

```
-- ================================================================================
-- Coinzy MVP #8 — Identify funnel conversion (raw events)
-- Open: Identify_bottom_nav ∪ Identify_home (CameraScreen / HomeScreen)
-- Photo: photo_clicked_1/2 / Photo_clicked  ·  Submit: photos_submitted / Identification_done
-- ================================================================================

WITH bounds AS (
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(
      user_id,
      (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
      user_pseudo_id
    ) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
)
SELECT
  event_date,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'Identify_bottom_nav', 'Identify_home'
  ) THEN resolved_user_id END) AS identify_open,
  COUNT(DISTINCT CASE WHEN event_name_base = 'camera_permission_granted' THEN resolved_user_id END)
    AS camera_granted,
  COUNT(DISTINCT CASE WHEN event_name_base = 'camera_permission_denied' THEN resolved_user_id END)
    AS camera_denied,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'photo_clicked_1', 'photo_clicked_2', 'Photo_clicked'
```

```
  ) THEN resolved_user_id END) AS photo_captured,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'photo_submit_button', 'photos_submitted', 'Identification_done'
  ) THEN resolved_user_id END) AS submit,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'identification_done_success', 'Identification_done_success'
  ) THEN resolved_user_id END) AS success,
  COUNT(DISTINCT CASE WHEN event_name_base IN (
    'identification_done_failure', 'Identification_done_failure',
    'Identification_failed', 'Identification_unsuccessful'
  ) THEN resolved_user_id END) AS failure,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'identification_done_success', 'Identification_done_success'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Identify_bottom_nav', 'Identify_home'
    ) THEN resolved_user_id END)
  ) AS open_to_success_rate,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'photo_clicked_1', 'photo_clicked_2', 'Photo_clicked'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'Identify_bottom_nav', 'Identify_home'
    ) THEN resolved_user_id END)
  ) AS open_to_photo_rate,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'identification_done_success', 'Identification_done_success'
    ) THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'photo_submit_button', 'photos_submitted', 'Identification_done'
    ) THEN resolved_user_id END)
  ) AS submit_to_success_rate,
  SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN event_name_base = 'camera_permission_granted' THEN resolved_user_id END),
    COUNT(DISTINCT CASE WHEN event_name_base IN (
      'camera_permission_granted', 'camera_permission_denied'
    ) THEN resolved_user_id END)
  ) AS camera_permission_grant_rate
FROM base
GROUP BY event_date
ORDER BY event_date;
```

Banknote

`sql/dashboard/product/08_identify_funnel_conversion.sql → v_identify_metrics (see section 3)`

Banknote product SQL selects users_identify_open / users_success from `v_identify_metrics` (full view SQL is in section 3).

```
-- ================================================================================
-- MVP #8 — Identify funnel conversion (open → success)
-- Covers missing "funnels" ask: where Identify breaks.
-- ================================================================================

SELECT
  event_date,
  SUM(users_identify_open) AS identify_open,
  SUM(users_camera_granted) AS camera_granted,
  SUM(users_camera_denied) AS camera_denied,
  SUM(users_photo_captured) AS photo_captured,
  SUM(users_submit) AS submit,
  SUM(users_success) AS success,
  SUM(users_failure) AS failure,
  SAFE_DIVIDE(SUM(users_success), SUM(users_identify_open)) AS open_to_success_rate,
  SAFE_DIVIDE(SUM(users_photo_captured), SUM(users_identify_open)) AS open_to_photo_rate,
  SAFE_DIVIDE(SUM(users_success), SUM(users_submit)) AS submit_to_success_rate,
  SAFE_DIVIDE(SUM(users_camera_granted), SUM(users_camera_granted) + SUM(users_camera_denied))
    AS camera_permission_grant_rate
FROM `{PROJECT}.{DATASET}.v_identify_metrics`
WHERE event_date BETWEEN {{start_date}} AND {{end_date}}
  [[AND country = {{country}}]]
  [[AND platform = {{platform}}]]
```

```
GROUP BY event_date
ORDER BY event_date;
```

Funnels — Identify (all)

> Generated in funnel-registry.js (not a .sql file). One events_* scan. Distinct users per step. YYYYMMDD in _TABLE_SUFFIX is replaced by the dashboard date filter.
>
> **Scan · bottom nav** is this same query with entry events = `Identify_bottom_nav` only.
> **Scan · home / banner** is this same query with entry events = `Identify_home` only.

Banknote

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Identify_bottom_nav', 'Identify_home')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Identification_screen', 'photo_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Camera_permission_popup')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('camera_permission_granted')) AS hits_3,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('camer_permission_denied',
'camera_permission_denied')) AS hits_4,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_clicked_1', 'photo_uploaded_1')) AS hits_5,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_clicked_1')) AS hits_6,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_uploaded_1')) AS hits_7,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_cropping_screen_1')) AS hits_8,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_crop_tick_1')) AS hits_9,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_clicked_2', 'photo_uploaded_2')) AS hits_10,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_clicked_2')) AS hits_11,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_uploaded_2')) AS hits_12,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_cropping_screen_2')) AS hits_13,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_crop_tick_2')) AS hits_14,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Photo_clicked')) AS hits_15,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_submit_button', 'photos_submitted')) AS
hits_16,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Identified_limit_reached',
'identiifcation_limit_exceeded', 'scan_quota_exhausted')) AS hits_17,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_done_success',
'Identification_done_success')) AS hits_18,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_done_failure',
'Identification_done_failure')) AS hits_19,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_top5_matches')) AS hits_20,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_view_all')) AS hits_21,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_all_options_screen')) AS hits_22,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('idetnification_option_chosen',
'identification_option_chosen')) AS hits_23,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_details_screen',
'banknote_details_identification')) AS hits_24,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Added_to_collection_identified',
'Added_to_collection_owned')) AS hits_25
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Identify_bottom_nav', 'Identify_home', 'Identification_screen',
'photo_screen', 'Camera_permission_popup', 'camera_permission_granted', 'camer_permission_denied',
'camera_permission_denied', 'photo_clicked_1', 'photo_uploaded_1', 'photo_cropping_screen_1', 'photo_crop_tick_1',
'photo_clicked_2', 'photo_uploaded_2', 'photo_cropping_screen_2', 'photo_crop_tick_2', 'Photo_clicked',
'photo_submit_button', 'photos_submitted', 'Identified_limit_reached', 'identiifcation_limit_exceeded',
'scan_quota_exhausted', 'identification_done_success', 'Identification_done_success', 'identification_done_failure',
'Identification_done_failure', 'identification_top5_matches', 'identification_view_all',
'identification_all_options_screen', 'idetnification_option_chosen', 'identification_option_chosen',
'identification_details_screen', 'banknote_details_identification', 'Added_to_collection_identified',
'Added_to_collection_owned'))
  GROUP BY uid
),
agg AS (
```

```
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3,
    COUNTIF(hits_4 > 0) AS users_4,
    COUNTIF(hits_4 = 1) AS once_4,
    COUNTIF(hits_4 >= 2) AS repeat_4,
    SUM(hits_4) AS hits_4,
    COUNTIF(hits_5 > 0) AS users_5,
    COUNTIF(hits_5 = 1) AS once_5,
    COUNTIF(hits_5 >= 2) AS repeat_5,
    SUM(hits_5) AS hits_5,
    COUNTIF(hits_6 > 0) AS users_6,
    COUNTIF(hits_6 = 1) AS once_6,
    COUNTIF(hits_6 >= 2) AS repeat_6,
    SUM(hits_6) AS hits_6,
    COUNTIF(hits_7 > 0) AS users_7,
    COUNTIF(hits_7 = 1) AS once_7,
    COUNTIF(hits_7 >= 2) AS repeat_7,
    SUM(hits_7) AS hits_7,
    COUNTIF(hits_8 > 0) AS users_8,
    COUNTIF(hits_8 = 1) AS once_8,
    COUNTIF(hits_8 >= 2) AS repeat_8,
    SUM(hits_8) AS hits_8,
    COUNTIF(hits_9 > 0) AS users_9,
    COUNTIF(hits_9 = 1) AS once_9,
    COUNTIF(hits_9 >= 2) AS repeat_9,
    SUM(hits_9) AS hits_9,
    COUNTIF(hits_10 > 0) AS users_10,
    COUNTIF(hits_10 = 1) AS once_10,
    COUNTIF(hits_10 >= 2) AS repeat_10,
    SUM(hits_10) AS hits_10,
    COUNTIF(hits_11 > 0) AS users_11,
    COUNTIF(hits_11 = 1) AS once_11,
    COUNTIF(hits_11 >= 2) AS repeat_11,
    SUM(hits_11) AS hits_11,
    COUNTIF(hits_12 > 0) AS users_12,
    COUNTIF(hits_12 = 1) AS once_12,
    COUNTIF(hits_12 >= 2) AS repeat_12,
    SUM(hits_12) AS hits_12,
    COUNTIF(hits_13 > 0) AS users_13,
    COUNTIF(hits_13 = 1) AS once_13,
    COUNTIF(hits_13 >= 2) AS repeat_13,
    SUM(hits_13) AS hits_13,
    COUNTIF(hits_14 > 0) AS users_14,
    COUNTIF(hits_14 = 1) AS once_14,
    COUNTIF(hits_14 >= 2) AS repeat_14,
    SUM(hits_14) AS hits_14,
    COUNTIF(hits_15 > 0) AS users_15,
    COUNTIF(hits_15 = 1) AS once_15,
    COUNTIF(hits_15 >= 2) AS repeat_15,
    SUM(hits_15) AS hits_15,
    COUNTIF(hits_16 > 0) AS users_16,
    COUNTIF(hits_16 = 1) AS once_16,
    COUNTIF(hits_16 >= 2) AS repeat_16,
    SUM(hits_16) AS hits_16,
    COUNTIF(hits_17 > 0) AS users_17,
    COUNTIF(hits_17 = 1) AS once_17,
    COUNTIF(hits_17 >= 2) AS repeat_17,
    SUM(hits_17) AS hits_17,
    COUNTIF(hits_18 > 0) AS users_18,
    COUNTIF(hits_18 = 1) AS once_18,
    COUNTIF(hits_18 >= 2) AS repeat_18,
```

```
    SUM(hits_18) AS hits_18,
    COUNTIF(hits_19 > 0) AS users_19,
    COUNTIF(hits_19 = 1) AS once_19,
    COUNTIF(hits_19 >= 2) AS repeat_19,
    SUM(hits_19) AS hits_19,
    COUNTIF(hits_20 > 0) AS users_20,
    COUNTIF(hits_20 = 1) AS once_20,
    COUNTIF(hits_20 >= 2) AS repeat_20,
    SUM(hits_20) AS hits_20,
    COUNTIF(hits_21 > 0) AS users_21,
    COUNTIF(hits_21 = 1) AS once_21,
    COUNTIF(hits_21 >= 2) AS repeat_21,
    SUM(hits_21) AS hits_21,
    COUNTIF(hits_22 > 0) AS users_22,
    COUNTIF(hits_22 = 1) AS once_22,
    COUNTIF(hits_22 >= 2) AS repeat_22,
    SUM(hits_22) AS hits_22,
    COUNTIF(hits_23 > 0) AS users_23,
    COUNTIF(hits_23 = 1) AS once_23,
    COUNTIF(hits_23 >= 2) AS repeat_23,
    SUM(hits_23) AS hits_23,
    COUNTIF(hits_24 > 0) AS users_24,
    COUNTIF(hits_24 = 1) AS once_24,
    COUNTIF(hits_24 >= 2) AS repeat_24,
    SUM(hits_24) AS hits_24,
    COUNTIF(hits_25 > 0) AS users_25,
    COUNTIF(hits_25 = 1) AS once_25,
    COUNTIF(hits_25 >= 2) AS repeat_25,
    SUM(hits_25) AS hits_25
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'entry' AS step_id,
    'Identify entry (nav ∪ home)' AS step_label,
    'Identify_bottom_nav, Identify_home' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'camera' AS step_id,
    'Camera / photo screen' AS step_label,
    'Identification_screen, photo_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'permission_popup' AS step_id,
    'Camera permission popup' AS step_label,
    'Camera_permission_popup' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
```

```
UNION ALL

SELECT
  4 AS step_order,
  'permission_granted' AS step_id,
  'Camera permission granted' AS step_label,
  'camera_permission_granted' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_3 AS users,
  once_3 AS once_users,
  repeat_3 AS repeat_users,
  hits_3 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  5 AS step_order,
  'permission_denied' AS step_id,
  'Camera permission denied' AS step_label,
  'camer_permission_denied, camera_permission_denied' AS event_names,
  FALSE AS is_core,
  TRUE AS is_drop,
  users_4 AS users,
  once_4 AS once_users,
  repeat_4 AS repeat_users,
  hits_4 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  6 AS step_order,
  'photo_1' AS step_id,
  'First image (camera or gallery)' AS step_label,
  'photo_clicked_1, photo_uploaded_1' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_5 AS users,
  once_5 AS once_users,
  repeat_5 AS repeat_users,
  hits_5 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  7 AS step_order,
  'photo_click_1' AS step_id,
  'First image · camera' AS step_label,
  'photo_clicked_1' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_6 AS users,
  once_6 AS once_users,
  repeat_6 AS repeat_users,
  hits_6 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  8 AS step_order,
  'photo_upload_1' AS step_id,
  'First image · gallery' AS step_label,
  'photo_uploaded_1' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_7 AS users,
  once_7 AS once_users,
  repeat_7 AS repeat_users,
  hits_7 AS hits,
  dau
FROM agg
UNION ALL
```

```
  SELECT
    9 AS step_order,
    'crop_1' AS step_id,
    'Crop first image' AS step_label,
    'photo_cropping_screen_1' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_8 AS users,
    once_8 AS once_users,
    repeat_8 AS repeat_users,
    hits_8 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    10 AS step_order,
    'crop_confirm_1' AS step_id,
    'First crop confirmed' AS step_label,
    'photo_crop_tick_1' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_9 AS users,
    once_9 AS once_users,
    repeat_9 AS repeat_users,
    hits_9 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    11 AS step_order,
    'photo_2' AS step_id,
    'Second image (camera or gallery)' AS step_label,
    'photo_clicked_2, photo_uploaded_2' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_10 AS users,
    once_10 AS once_users,
    repeat_10 AS repeat_users,
    hits_10 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    12 AS step_order,
    'photo_click_2' AS step_id,
    'Second image · camera' AS step_label,
    'photo_clicked_2' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_11 AS users,
    once_11 AS once_users,
    repeat_11 AS repeat_users,
    hits_11 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    13 AS step_order,
    'photo_upload_2' AS step_id,
    'Second image · gallery' AS step_label,
    'photo_uploaded_2' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_12 AS users,
    once_12 AS once_users,
    repeat_12 AS repeat_users,
    hits_12 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    14 AS step_order,
```

```
  'crop_2' AS step_id,
  'Crop second image' AS step_label,
  'photo_cropping_screen_2' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_13 AS users,
  once_13 AS once_users,
  repeat_13 AS repeat_users,
  hits_13 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  15 AS step_order,
  'crop_confirm_2' AS step_id,
  'Second crop confirmed' AS step_label,
  'photo_crop_tick_2' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_14 AS users,
  once_14 AS once_users,
  repeat_14 AS repeat_users,
  hits_14 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  16 AS step_order,
  'photo_unspecified' AS step_id,
  'Photo (unspecified)' AS step_label,
  'Photo_clicked' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_15 AS users,
  once_15 AS once_users,
  repeat_15 AS repeat_users,
  hits_15 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  17 AS step_order,
  'submit' AS step_id,
  'Submit photos' AS step_label,
  'photo_submit_button, photos_submitted' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_16 AS users,
  once_16 AS once_users,
  repeat_16 AS repeat_users,
  hits_16 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  18 AS step_order,
  'quota_block' AS step_id,
  'Quota / limit block' AS step_label,
  'Identified_limit_reached, identiifcation_limit_exceeded, scan_quota_exhausted' AS event_names,
  FALSE AS is_core,
  TRUE AS is_drop,
  users_17 AS users,
  once_17 AS once_users,
  repeat_17 AS repeat_users,
  hits_17 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  19 AS step_order,
  'success' AS step_id,
  'Identification success' AS step_label,
```

```
  'identification_done_success, Identification_done_success' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_18 AS users,
  once_18 AS once_users,
  repeat_18 AS repeat_users,
  hits_18 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  20 AS step_order,
  'failure' AS step_id,
  'Identification failure' AS step_label,
  'identification_done_failure, Identification_done_failure' AS event_names,
  FALSE AS is_core,
  TRUE AS is_drop,
  users_19 AS users,
  once_19 AS once_users,
  repeat_19 AS repeat_users,
  hits_19 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  21 AS step_order,
  'top_matches' AS step_id,
  'Top matches screen' AS step_label,
  'identification_top5_matches' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_20 AS users,
  once_20 AS once_users,
  repeat_20 AS repeat_users,
  hits_20 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  22 AS step_order,
  'view_all' AS step_id,
  'View all other options' AS step_label,
  'identification_view_all' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_21 AS users,
  once_21 AS once_users,
  repeat_21 AS repeat_users,
  hits_21 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  23 AS step_order,
  'all_options' AS step_id,
  'All options screen' AS step_label,
  'identification_all_options_screen' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_22 AS users,
  once_22 AS once_users,
  repeat_22 AS repeat_users,
  hits_22 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  24 AS step_order,
  'option_chosen' AS step_id,
  'Option chosen' AS step_label,
  'idetnification_option_chosen, identification_option_chosen' AS event_names,
  FALSE AS is_core,
```

```
    FALSE AS is_drop,
    users_23 AS users,
    once_23 AS once_users,
    repeat_23 AS repeat_users,
    hits_23 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    25 AS step_order,
    'details' AS step_id,
    'ID / banknote details' AS step_label,
    'identification_details_screen, banknote_details_identification' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_24 AS users,
    once_24 AS once_users,
    repeat_24 AS repeat_users,
    hits_24 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    26 AS step_order,
    'add_collection' AS step_id,
    'Add to collection after ID' AS step_label,
    'Added_to_collection_identified, Added_to_collection_owned' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_25 AS users,
    once_25 AS once_users,
    repeat_25 AS repeat_users,
    hits_25 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Coinzy

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Identify_bottom_nav', 'Identify_home')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Identification_screen', 'photo_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Camera_permission_popup')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('camera_permission_granted')) AS hits_3,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('camer_permission_denied',
'camera_permission_denied')) AS hits_4,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_clicked_1', 'photo_uploaded_1')) AS hits_5,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_clicked_1')) AS hits_6,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_uploaded_1')) AS hits_7,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_cropping_screen_0')) AS hits_8,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_crop_tick_0')) AS hits_9,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_clicked_2', 'photo_uploaded_2')) AS hits_10,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_clicked_2')) AS hits_11,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_uploaded_2')) AS hits_12,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_cropping_screen_1')) AS hits_13,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_crop_tick_1')) AS hits_14,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Photo_clicked')) AS hits_15,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_cropping_screen_2')) AS hits_16,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_crop_tick_2')) AS hits_17,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('photo_submit_button', 'photos_submitted',
'Identification_done')) AS hits_18,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Identified_limit_reached',
'identiifcation_limit_exceeded', 'free_scan_limit_exceeded', 'free_scan_blocked', 'free_scan_success_quota_exhausted',
'free_scan_fail_quota_exhausted', 'Identification_unsuccessful_limit_reached')) AS hits_19,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_done_success',
'Identification_done_success')) AS hits_20,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_done_failure',
'Identification_done_failure', 'Identification_failed', 'Identification_unsuccessful')) AS hits_21,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_view_all')) AS hits_22,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_all_options_screen')) AS hits_23,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('idetnification_option_chosen',
'identification_option_chosen')) AS hits_24,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('identification_details_screen',
'Coin_details_identification')) AS hits_25,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Added_to_collection_identified',
'Added_to_collection_owned', 'Added _to_collection_owned')) AS hits_26
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Identify_bottom_nav', 'Identify_home', 'Identification_screen',
'photo_screen', 'Camera_permission_popup', 'camera_permission_granted', 'camer_permission_denied',
'camera_permission_denied', 'photo_clicked_1', 'photo_uploaded_1', 'photo_cropping_screen_0', 'photo_crop_tick_0',
'photo_clicked_2', 'photo_uploaded_2', 'photo_cropping_screen_1', 'photo_crop_tick_1', 'Photo_clicked',
'photo_cropping_screen_2', 'photo_crop_tick_2', 'photo_submit_button', 'photos_submitted', 'Identification_done',
'Identified_limit_reached', 'identiifcation_limit_exceeded', 'free_scan_limit_exceeded', 'free_scan_blocked',
'free_scan_success_quota_exhausted', 'free_scan_fail_quota_exhausted', 'Identification_unsuccessful_limit_reached',
'identification_done_success', 'Identification_done_success', 'identification_done_failure',
'Identification_done_failure', 'Identification_failed', 'Identification_unsuccessful', 'identification_view_all',
'identification_all_options_screen', 'idetnification_option_chosen', 'identification_option_chosen',
'identification_details_screen', 'Coin_details_identification', 'Added_to_collection_identified',
'Added_to_collection_owned', 'Added _to_collection_owned'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
```

```
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3,
    COUNTIF(hits_4 > 0) AS users_4,
    COUNTIF(hits_4 = 1) AS once_4,
    COUNTIF(hits_4 >= 2) AS repeat_4,
    SUM(hits_4) AS hits_4,
    COUNTIF(hits_5 > 0) AS users_5,
    COUNTIF(hits_5 = 1) AS once_5,
    COUNTIF(hits_5 >= 2) AS repeat_5,
    SUM(hits_5) AS hits_5,
    COUNTIF(hits_6 > 0) AS users_6,
    COUNTIF(hits_6 = 1) AS once_6,
    COUNTIF(hits_6 >= 2) AS repeat_6,
    SUM(hits_6) AS hits_6,
    COUNTIF(hits_7 > 0) AS users_7,
    COUNTIF(hits_7 = 1) AS once_7,
    COUNTIF(hits_7 >= 2) AS repeat_7,
    SUM(hits_7) AS hits_7,
    COUNTIF(hits_8 > 0) AS users_8,
    COUNTIF(hits_8 = 1) AS once_8,
    COUNTIF(hits_8 >= 2) AS repeat_8,
    SUM(hits_8) AS hits_8,
    COUNTIF(hits_9 > 0) AS users_9,
    COUNTIF(hits_9 = 1) AS once_9,
    COUNTIF(hits_9 >= 2) AS repeat_9,
    SUM(hits_9) AS hits_9,
    COUNTIF(hits_10 > 0) AS users_10,
    COUNTIF(hits_10 = 1) AS once_10,
    COUNTIF(hits_10 >= 2) AS repeat_10,
    SUM(hits_10) AS hits_10,
    COUNTIF(hits_11 > 0) AS users_11,
    COUNTIF(hits_11 = 1) AS once_11,
    COUNTIF(hits_11 >= 2) AS repeat_11,
    SUM(hits_11) AS hits_11,
    COUNTIF(hits_12 > 0) AS users_12,
    COUNTIF(hits_12 = 1) AS once_12,
    COUNTIF(hits_12 >= 2) AS repeat_12,
    SUM(hits_12) AS hits_12,
    COUNTIF(hits_13 > 0) AS users_13,
    COUNTIF(hits_13 = 1) AS once_13,
    COUNTIF(hits_13 >= 2) AS repeat_13,
    SUM(hits_13) AS hits_13,
    COUNTIF(hits_14 > 0) AS users_14,
    COUNTIF(hits_14 = 1) AS once_14,
    COUNTIF(hits_14 >= 2) AS repeat_14,
    SUM(hits_14) AS hits_14,
    COUNTIF(hits_15 > 0) AS users_15,
    COUNTIF(hits_15 = 1) AS once_15,
    COUNTIF(hits_15 >= 2) AS repeat_15,
    SUM(hits_15) AS hits_15,
    COUNTIF(hits_16 > 0) AS users_16,
    COUNTIF(hits_16 = 1) AS once_16,
    COUNTIF(hits_16 >= 2) AS repeat_16,
    SUM(hits_16) AS hits_16,
    COUNTIF(hits_17 > 0) AS users_17,
    COUNTIF(hits_17 = 1) AS once_17,
    COUNTIF(hits_17 >= 2) AS repeat_17,
    SUM(hits_17) AS hits_17,
    COUNTIF(hits_18 > 0) AS users_18,
    COUNTIF(hits_18 = 1) AS once_18,
    COUNTIF(hits_18 >= 2) AS repeat_18,
    SUM(hits_18) AS hits_18,
    COUNTIF(hits_19 > 0) AS users_19,
    COUNTIF(hits_19 = 1) AS once_19,
    COUNTIF(hits_19 >= 2) AS repeat_19,
    SUM(hits_19) AS hits_19,
    COUNTIF(hits_20 > 0) AS users_20,
    COUNTIF(hits_20 = 1) AS once_20,
    COUNTIF(hits_20 >= 2) AS repeat_20,
    SUM(hits_20) AS hits_20,
    COUNTIF(hits_21 > 0) AS users_21,
    COUNTIF(hits_21 = 1) AS once_21,
    COUNTIF(hits_21 >= 2) AS repeat_21,
    SUM(hits_21) AS hits_21,
```

```
    COUNTIF(hits_22 > 0) AS users_22,
    COUNTIF(hits_22 = 1) AS once_22,
    COUNTIF(hits_22 >= 2) AS repeat_22,
    SUM(hits_22) AS hits_22,
    COUNTIF(hits_23 > 0) AS users_23,
    COUNTIF(hits_23 = 1) AS once_23,
    COUNTIF(hits_23 >= 2) AS repeat_23,
    SUM(hits_23) AS hits_23,
    COUNTIF(hits_24 > 0) AS users_24,
    COUNTIF(hits_24 = 1) AS once_24,
    COUNTIF(hits_24 >= 2) AS repeat_24,
    SUM(hits_24) AS hits_24,
    COUNTIF(hits_25 > 0) AS users_25,
    COUNTIF(hits_25 = 1) AS once_25,
    COUNTIF(hits_25 >= 2) AS repeat_25,
    SUM(hits_25) AS hits_25,
    COUNTIF(hits_26 > 0) AS users_26,
    COUNTIF(hits_26 = 1) AS once_26,
    COUNTIF(hits_26 >= 2) AS repeat_26,
    SUM(hits_26) AS hits_26
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'entry' AS step_id,
    'Identify entry (nav ∪ home)' AS step_label,
    'Identify_bottom_nav, Identify_home' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'camera' AS step_id,
    'Camera / photo screen' AS step_label,
    'Identification_screen, photo_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'permission_popup' AS step_id,
    'Camera permission popup' AS step_label,
    'Camera_permission_popup' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    4 AS step_order,
    'permission_granted' AS step_id,
    'Camera permission granted' AS step_label,
    'camera_permission_granted' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
```

```
  users_3 AS users,
  once_3 AS once_users,
  repeat_3 AS repeat_users,
  hits_3 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  5 AS step_order,
  'permission_denied' AS step_id,
  'Camera permission denied' AS step_label,
  'camer_permission_denied, camera_permission_denied' AS event_names,
  FALSE AS is_core,
  TRUE AS is_drop,
  users_4 AS users,
  once_4 AS once_users,
  repeat_4 AS repeat_users,
  hits_4 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  6 AS step_order,
  'photo_1' AS step_id,
  'First image (camera or gallery)' AS step_label,
  'photo_clicked_1, photo_uploaded_1' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_5 AS users,
  once_5 AS once_users,
  repeat_5 AS repeat_users,
  hits_5 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  7 AS step_order,
  'photo_click_1' AS step_id,
  'First image · camera' AS step_label,
  'photo_clicked_1' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_6 AS users,
  once_6 AS once_users,
  repeat_6 AS repeat_users,
  hits_6 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  8 AS step_order,
  'photo_upload_1' AS step_id,
  'First image · gallery' AS step_label,
  'photo_uploaded_1' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_7 AS users,
  once_7 AS once_users,
  repeat_7 AS repeat_users,
  hits_7 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  9 AS step_order,
  'crop_1' AS step_id,
  'Crop first image' AS step_label,
  'photo_cropping_screen_0' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_8 AS users,
  once_8 AS once_users,
```

```
  repeat_8 AS repeat_users,
  hits_8 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  10 AS step_order,
  'crop_confirm_1' AS step_id,
  'First crop confirmed' AS step_label,
  'photo_crop_tick_0' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_9 AS users,
  once_9 AS once_users,
  repeat_9 AS repeat_users,
  hits_9 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  11 AS step_order,
  'photo_2' AS step_id,
  'Second image (camera or gallery)' AS step_label,
  'photo_clicked_2, photo_uploaded_2' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_10 AS users,
  once_10 AS once_users,
  repeat_10 AS repeat_users,
  hits_10 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  12 AS step_order,
  'photo_click_2' AS step_id,
  'Second image · camera' AS step_label,
  'photo_clicked_2' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_11 AS users,
  once_11 AS once_users,
  repeat_11 AS repeat_users,
  hits_11 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  13 AS step_order,
  'photo_upload_2' AS step_id,
  'Second image · gallery' AS step_label,
  'photo_uploaded_2' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_12 AS users,
  once_12 AS once_users,
  repeat_12 AS repeat_users,
  hits_12 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  14 AS step_order,
  'crop_2' AS step_id,
  'Crop second image' AS step_label,
  'photo_cropping_screen_1' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_13 AS users,
  once_13 AS once_users,
  repeat_13 AS repeat_users,
  hits_13 AS hits,
```

```
  dau
FROM agg
UNION ALL

SELECT
  15 AS step_order,
  'crop_confirm_2' AS step_id,
  'Second crop confirmed' AS step_label,
  'photo_crop_tick_1' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_14 AS users,
  once_14 AS once_users,
  repeat_14 AS repeat_users,
  hits_14 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  16 AS step_order,
  'photo_unspecified' AS step_id,
  'Photo (unspecified)' AS step_label,
  'Photo_clicked' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_15 AS users,
  once_15 AS once_users,
  repeat_15 AS repeat_users,
  hits_15 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  17 AS step_order,
  'crop_extra' AS step_id,
  'Crop (slot 3, unused)' AS step_label,
  'photo_cropping_screen_2' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_16 AS users,
  once_16 AS once_users,
  repeat_16 AS repeat_users,
  hits_16 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  18 AS step_order,
  'crop_confirm_extra' AS step_id,
  'Crop confirm (slot 3, unused)' AS step_label,
  'photo_crop_tick_2' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_17 AS users,
  once_17 AS once_users,
  repeat_17 AS repeat_users,
  hits_17 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  19 AS step_order,
  'submit' AS step_id,
  'Submit photos' AS step_label,
  'photo_submit_button, photos_submitted, Identification_done' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_18 AS users,
  once_18 AS once_users,
  repeat_18 AS repeat_users,
  hits_18 AS hits,
  dau
FROM agg
```

```
  UNION ALL

  SELECT
    20 AS step_order,
    'quota_block' AS step_id,
    'Quota / limit block' AS step_label,
    'Identified_limit_reached, identiifcation_limit_exceeded, free_scan_limit_exceeded, free_scan_blocked,
free_scan_success_quota_exhausted, free_scan_fail_quota_exhausted, Identification_unsuccessful_limit_reached' AS
event_names,
    FALSE AS is_core,
    TRUE AS is_drop,
    users_19 AS users,
    once_19 AS once_users,
    repeat_19 AS repeat_users,
    hits_19 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    21 AS step_order,
    'success' AS step_id,
    'Identification success' AS step_label,
    'identification_done_success, Identification_done_success' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_20 AS users,
    once_20 AS once_users,
    repeat_20 AS repeat_users,
    hits_20 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    22 AS step_order,
    'failure' AS step_id,
    'Identification failure' AS step_label,
    'identification_done_failure, Identification_done_failure, Identification_failed, Identification_unsuccessful' AS
event_names,
    FALSE AS is_core,
    TRUE AS is_drop,
    users_21 AS users,
    once_21 AS once_users,
    repeat_21 AS repeat_users,
    hits_21 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    23 AS step_order,
    'view_all' AS step_id,
    'View all other options' AS step_label,
    'identification_view_all' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_22 AS users,
    once_22 AS once_users,
    repeat_22 AS repeat_users,
    hits_22 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    24 AS step_order,
    'all_options' AS step_id,
    'All options / multi-coin match' AS step_label,
    'identification_all_options_screen' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_23 AS users,
    once_23 AS once_users,
    repeat_23 AS repeat_users,
    hits_23 AS hits,
    dau
```

```
  FROM agg
  UNION ALL

  SELECT
    25 AS step_order,
    'option_chosen' AS step_id,
    'Option chosen (typo event name in app)' AS step_label,
    'idetnification_option_chosen, identification_option_chosen' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_24 AS users,
    once_24 AS once_users,
    repeat_24 AS repeat_users,
    hits_24 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    26 AS step_order,
    'details' AS step_id,
    'ID / coin details after ID' AS step_label,
    'identification_details_screen, Coin_details_identification' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_25 AS users,
    once_25 AS once_users,
    repeat_25 AS repeat_users,
    hits_25 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    27 AS step_order,
    'add_collection' AS step_id,
    'Add to collection after ID' AS step_label,
    'Added_to_collection_identified, Added_to_collection_owned, Added _to_collection_owned' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_26 AS users,
    once_26 AS once_users,
    repeat_26 AS repeat_users,
    hits_26 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
```

```
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Funnels — Catalogue · Collection · Global · Marketplace · Feed · Paywall · Expert

> Each tab is its own generated query. Marketplace excludes Feed. Expert is Coinzy only.

Banknote · catalogue (all)

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('private_collection_bottom_nav')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Collection_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Collection_clicked')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Sub_collection_Screen')) AS hits_3,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('sub_collection_item')) AS hits_4,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('banknote_details_collection')) AS hits_5,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Global_catalogue', 'View_all_button_global')) AS
hits_6,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Global_catalogue_screen')) AS hits_7,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('global_catalogue_item')) AS hits_8,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('banknote_details_global')) AS hits_9,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Collection_screen', 'Global_catalogue_screen',
'Collection_open', 'collection_open', 'Collection', 'My_collection')) AS hits_10
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('private_collection_bottom_nav', 'Collection_screen',
'Collection_clicked', 'Sub_collection_Screen', 'sub_collection_item', 'banknote_details_collection', 'Global_catalogue',
'View_all_button_global', 'Global_catalogue_screen', 'global_catalogue_item', 'banknote_details_global',
'Collection_open', 'collection_open', 'Collection', 'My_collection'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3,
    COUNTIF(hits_4 > 0) AS users_4,
    COUNTIF(hits_4 = 1) AS once_4,
    COUNTIF(hits_4 >= 2) AS repeat_4,
    SUM(hits_4) AS hits_4,
    COUNTIF(hits_5 > 0) AS users_5,
    COUNTIF(hits_5 = 1) AS once_5,
    COUNTIF(hits_5 >= 2) AS repeat_5,
    SUM(hits_5) AS hits_5,
    COUNTIF(hits_6 > 0) AS users_6,
    COUNTIF(hits_6 = 1) AS once_6,
    COUNTIF(hits_6 >= 2) AS repeat_6,
```

```
    SUM(hits_6) AS hits_6,
    COUNTIF(hits_7 > 0) AS users_7,
    COUNTIF(hits_7 = 1) AS once_7,
    COUNTIF(hits_7 >= 2) AS repeat_7,
    SUM(hits_7) AS hits_7,
    COUNTIF(hits_8 > 0) AS users_8,
    COUNTIF(hits_8 = 1) AS once_8,
    COUNTIF(hits_8 >= 2) AS repeat_8,
    SUM(hits_8) AS hits_8,
    COUNTIF(hits_9 > 0) AS users_9,
    COUNTIF(hits_9 = 1) AS once_9,
    COUNTIF(hits_9 >= 2) AS repeat_9,
    SUM(hits_9) AS hits_9,
    COUNTIF(hits_10 > 0) AS users_10,
    COUNTIF(hits_10 = 1) AS once_10,
    COUNTIF(hits_10 >= 2) AS repeat_10,
    SUM(hits_10) AS hits_10
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'collection_tab' AS step_id,
    'Collection bottom nav' AS step_label,
    'private_collection_bottom_nav' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'collection_screen' AS step_id,
    'Collection screen' AS step_label,
    'Collection_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'collection_clicked' AS step_id,
    'Open collection card' AS step_label,
    'Collection_clicked' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    4 AS step_order,
    'sub_collection' AS step_id,
    'Sub-collection screen' AS step_label,
    'Sub_collection_Screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_3 AS users,
    once_3 AS once_users,
    repeat_3 AS repeat_users,
```

```
  hits_3 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  5 AS step_order,
  'sub_item' AS step_id,
  'Sub-collection item tap' AS step_label,
  'sub_collection_item' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_4 AS users,
  once_4 AS once_users,
  repeat_4 AS repeat_users,
  hits_4 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  6 AS step_order,
  'details_collection' AS step_id,
  'Details from collection' AS step_label,
  'banknote_details_collection' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_5 AS users,
  once_5 AS once_users,
  repeat_5 AS repeat_users,
  hits_5 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  7 AS step_order,
  'global_cta' AS step_id,
  'Global catalogue CTA' AS step_label,
  'Global_catalogue, View_all_button_global' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_6 AS users,
  once_6 AS once_users,
  repeat_6 AS repeat_users,
  hits_6 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  8 AS step_order,
  'global_screen' AS step_id,
  'Global catalogue screen' AS step_label,
  'Global_catalogue_screen' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_7 AS users,
  once_7 AS once_users,
  repeat_7 AS repeat_users,
  hits_7 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  9 AS step_order,
  'global_item' AS step_id,
  'Global catalogue item' AS step_label,
  'global_catalogue_item' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_8 AS users,
  once_8 AS once_users,
  repeat_8 AS repeat_users,
  hits_8 AS hits,
  dau
```

```
  FROM agg
  UNION ALL

  SELECT
    10 AS step_order,
    'details_global' AS step_id,
    'Details from global' AS step_label,
    'banknote_details_global' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_9 AS users,
    once_9 AS once_users,
    repeat_9 AS repeat_users,
    hits_9 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    11 AS step_order,
    'open_kpi' AS step_id,
    'Catalogue open (KPI union)' AS step_label,
    'Collection_screen, Global_catalogue_screen, Collection_open, collection_open, Collection, My_collection' AS
event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_10 AS users,
    once_10 AS once_users,
    repeat_10 AS repeat_users,
    hits_10 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Coinzy · catalogue (all)

```
WITH per_user AS (
  SELECT
```

```
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('collection_bottom_nav')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Collection_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Collection_clicked')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Sub_collection_Screen')) AS hits_3,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('sub_collection_item')) AS hits_4,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Coin_details_collection', 'Coin_details')) AS
hits_5,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Global_catalogue')) AS hits_6,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Global_catalogue_screen')) AS hits_7,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('global_catalogue_item')) AS hits_8,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Coin_details_global')) AS hits_9,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Collection_screen', 'Global_catalogue_screen')) AS
hits_10
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('collection_bottom_nav', 'Collection_screen',
'Collection_clicked', 'Sub_collection_Screen', 'sub_collection_item', 'Coin_details_collection', 'Coin_details',
'Global_catalogue', 'Global_catalogue_screen', 'global_catalogue_item', 'Coin_details_global'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3,
    COUNTIF(hits_4 > 0) AS users_4,
    COUNTIF(hits_4 = 1) AS once_4,
    COUNTIF(hits_4 >= 2) AS repeat_4,
    SUM(hits_4) AS hits_4,
    COUNTIF(hits_5 > 0) AS users_5,
    COUNTIF(hits_5 = 1) AS once_5,
    COUNTIF(hits_5 >= 2) AS repeat_5,
    SUM(hits_5) AS hits_5,
    COUNTIF(hits_6 > 0) AS users_6,
    COUNTIF(hits_6 = 1) AS once_6,
    COUNTIF(hits_6 >= 2) AS repeat_6,
    SUM(hits_6) AS hits_6,
    COUNTIF(hits_7 > 0) AS users_7,
    COUNTIF(hits_7 = 1) AS once_7,
    COUNTIF(hits_7 >= 2) AS repeat_7,
    SUM(hits_7) AS hits_7,
    COUNTIF(hits_8 > 0) AS users_8,
    COUNTIF(hits_8 = 1) AS once_8,
    COUNTIF(hits_8 >= 2) AS repeat_8,
    SUM(hits_8) AS hits_8,
    COUNTIF(hits_9 > 0) AS users_9,
    COUNTIF(hits_9 = 1) AS once_9,
    COUNTIF(hits_9 >= 2) AS repeat_9,
    SUM(hits_9) AS hits_9,
    COUNTIF(hits_10 > 0) AS users_10,
    COUNTIF(hits_10 = 1) AS once_10,
    COUNTIF(hits_10 >= 2) AS repeat_10,
    SUM(hits_10) AS hits_10
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (
```

```
SELECT
  1 AS step_order,
  'collection_tab' AS step_id,
  'Collection bottom nav' AS step_label,
  'collection_bottom_nav' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_0 AS users,
  once_0 AS once_users,
  repeat_0 AS repeat_users,
  hits_0 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  2 AS step_order,
  'collection_screen' AS step_id,
  'Collection screen' AS step_label,
  'Collection_screen' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_1 AS users,
  once_1 AS once_users,
  repeat_1 AS repeat_users,
  hits_1 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  3 AS step_order,
  'collection_clicked' AS step_id,
  'Open collection card' AS step_label,
  'Collection_clicked' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_2 AS users,
  once_2 AS once_users,
  repeat_2 AS repeat_users,
  hits_2 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  4 AS step_order,
  'sub_collection' AS step_id,
  'Sub-collection screen' AS step_label,
  'Sub_collection_Screen' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_3 AS users,
  once_3 AS once_users,
  repeat_3 AS repeat_users,
  hits_3 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  5 AS step_order,
  'sub_item' AS step_id,
  'Sub-collection item tap' AS step_label,
  'sub_collection_item' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_4 AS users,
  once_4 AS once_users,
  repeat_4 AS repeat_users,
  hits_4 AS hits,
  dau
FROM agg
UNION ALL

SELECT
```

```
  6 AS step_order,
  'details_collection' AS step_id,
  'Coin details from collection' AS step_label,
  'Coin_details_collection, Coin_details' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_5 AS users,
  once_5 AS once_users,
  repeat_5 AS repeat_users,
  hits_5 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  7 AS step_order,
  'global_cta' AS step_id,
  'Global catalogue CTA' AS step_label,
  'Global_catalogue' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_6 AS users,
  once_6 AS once_users,
  repeat_6 AS repeat_users,
  hits_6 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  8 AS step_order,
  'global_screen' AS step_id,
  'Global catalogue screen' AS step_label,
  'Global_catalogue_screen' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_7 AS users,
  once_7 AS once_users,
  repeat_7 AS repeat_users,
  hits_7 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  9 AS step_order,
  'global_item' AS step_id,
  'Global catalogue item' AS step_label,
  'global_catalogue_item' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_8 AS users,
  once_8 AS once_users,
  repeat_8 AS repeat_users,
  hits_8 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  10 AS step_order,
  'details_global' AS step_id,
  'Coin details from global' AS step_label,
  'Coin_details_global' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_9 AS users,
  once_9 AS once_users,
  repeat_9 AS repeat_users,
  hits_9 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  11 AS step_order,
  'open_kpi' AS step_id,
```

```
    'Catalogue open (KPI union)' AS step_label,
    'Collection_screen, Global_catalogue_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_10 AS users,
    once_10 AS once_users,
    repeat_10 AS repeat_users,
    hits_10 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Banknote · private collection

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('private_collection_bottom_nav')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Collection_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Collection_clicked')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Sub_collection_Screen')) AS hits_3,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('sub_collection_item')) AS hits_4,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('banknote_details_collection')) AS hits_5
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('private_collection_bottom_nav', 'Collection_screen',
'Collection_clicked', 'Sub_collection_Screen', 'sub_collection_item', 'banknote_details_collection'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
```

```
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3,
    COUNTIF(hits_4 > 0) AS users_4,
    COUNTIF(hits_4 = 1) AS once_4,
    COUNTIF(hits_4 >= 2) AS repeat_4,
    SUM(hits_4) AS hits_4,
    COUNTIF(hits_5 > 0) AS users_5,
    COUNTIF(hits_5 = 1) AS once_5,
    COUNTIF(hits_5 >= 2) AS repeat_5,
    SUM(hits_5) AS hits_5
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'collection_tab' AS step_id,
    'Collection bottom nav' AS step_label,
    'private_collection_bottom_nav' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'collection_screen' AS step_id,
    'Collection screen' AS step_label,
    'Collection_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'collection_clicked' AS step_id,
    'Open collection card' AS step_label,
    'Collection_clicked' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    4 AS step_order,
    'sub_collection' AS step_id,
    'Sub-collection screen' AS step_label,
    'Sub_collection_Screen' AS event_names,
```

```
    TRUE AS is_core,
    FALSE AS is_drop,
    users_3 AS users,
    once_3 AS once_users,
    repeat_3 AS repeat_users,
    hits_3 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    5 AS step_order,
    'sub_item' AS step_id,
    'Sub-collection item tap' AS step_label,
    'sub_collection_item' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_4 AS users,
    once_4 AS once_users,
    repeat_4 AS repeat_users,
    hits_4 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    6 AS step_order,
    'details_collection' AS step_id,
    'Details from collection' AS step_label,
    'banknote_details_collection' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_5 AS users,
    once_5 AS once_users,
    repeat_5 AS repeat_users,
    hits_5 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Coinzy · private collection

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('collection_bottom_nav')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Collection_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Collection_clicked')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Sub_collection_Screen')) AS hits_3,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('sub_collection_item')) AS hits_4,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Coin_details_collection', 'Coin_details')) AS hits_5
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('collection_bottom_nav', 'Collection_screen',
'Collection_clicked', 'Sub_collection_Screen', 'sub_collection_item', 'Coin_details_collection', 'Coin_details'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3,
    COUNTIF(hits_4 > 0) AS users_4,
    COUNTIF(hits_4 = 1) AS once_4,
    COUNTIF(hits_4 >= 2) AS repeat_4,
    SUM(hits_4) AS hits_4,
    COUNTIF(hits_5 > 0) AS users_5,
    COUNTIF(hits_5 = 1) AS once_5,
    COUNTIF(hits_5 >= 2) AS repeat_5,
    SUM(hits_5) AS hits_5
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'collection_tab' AS step_id,
    'Collection bottom nav' AS step_label,
    'collection_bottom_nav' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'collection_screen' AS step_id,
    'Collection screen' AS step_label,
    'Collection_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
```

```
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'collection_clicked' AS step_id,
    'Open collection card' AS step_label,
    'Collection_clicked' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    4 AS step_order,
    'sub_collection' AS step_id,
    'Sub-collection screen' AS step_label,
    'Sub_collection_Screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_3 AS users,
    once_3 AS once_users,
    repeat_3 AS repeat_users,
    hits_3 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    5 AS step_order,
    'sub_item' AS step_id,
    'Sub-collection item tap' AS step_label,
    'sub_collection_item' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_4 AS users,
    once_4 AS once_users,
    repeat_4 AS repeat_users,
    hits_4 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    6 AS step_order,
    'details_collection' AS step_id,
    'Coin details from collection' AS step_label,
    'Coin_details_collection, Coin_details' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_5 AS users,
    once_5 AS once_users,
    repeat_5 AS repeat_users,
    hits_5 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
```

```
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Banknote · global catalogue

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Global_catalogue', 'View_all_button_global')) AS
hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Global_catalogue_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('global_catalogue_item')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('banknote_details_global')) AS hits_3
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Global_catalogue', 'View_all_button_global',
'Global_catalogue_screen', 'global_catalogue_item', 'banknote_details_global'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'global_cta' AS step_id,
```

```
    'Global catalogue CTA' AS step_label,
    'Global_catalogue, View_all_button_global' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'global_screen' AS step_id,
    'Global catalogue screen' AS step_label,
    'Global_catalogue_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'global_item' AS step_id,
    'Global catalogue item' AS step_label,
    'global_catalogue_item' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    4 AS step_order,
    'details_global' AS step_id,
    'Details from global' AS step_label,
    'banknote_details_global' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_3 AS users,
    once_3 AS once_users,
    repeat_3 AS repeat_users,
    hits_3 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
```

```
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Coinzy · global catalogue

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Global_catalogue')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Global_catalogue_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('global_catalogue_item')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Coin_details_global')) AS hits_3
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Global_catalogue', 'Global_catalogue_screen',
'global_catalogue_item', 'Coin_details_global'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'global_cta' AS step_id,
    'Global catalogue CTA' AS step_label,
    'Global_catalogue' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL
```

```
  SELECT
    2 AS step_order,
    'global_screen' AS step_id,
    'Global catalogue screen' AS step_label,
    'Global_catalogue_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'global_item' AS step_id,
    'Global catalogue item' AS step_label,
    'global_catalogue_item' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    4 AS step_order,
    'details_global' AS step_id,
    'Coin details from global' AS step_label,
    'Coin_details_global' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_3 AS users,
    once_3 AS once_users,
    repeat_3 AS repeat_users,
    hits_3 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
```

```
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Banknote · marketplace

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Marketplace_bottom_nav', 'marketplace_bottom_nav'))
AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('marketplace_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('market_item_expolre')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('sale_Details_screen')) AS hits_3,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('market_contact', 'market_contact_button')) AS
hits_4,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('add_for_sale_button_marketplace',
'add_for_sale_details_scr_btn')) AS hits_5,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('market_add', 'market_contact_Add_for_sale')) AS
hits_6
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Marketplace_bottom_nav', 'marketplace_bottom_nav',
'marketplace_screen', 'market_item_expolre', 'sale_Details_screen', 'market_contact', 'market_contact_button',
'add_for_sale_button_marketplace', 'add_for_sale_details_scr_btn', 'market_add', 'market_contact_Add_for_sale'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3,
    COUNTIF(hits_4 > 0) AS users_4,
    COUNTIF(hits_4 = 1) AS once_4,
    COUNTIF(hits_4 >= 2) AS repeat_4,
    SUM(hits_4) AS hits_4,
    COUNTIF(hits_5 > 0) AS users_5,
    COUNTIF(hits_5 = 1) AS once_5,
    COUNTIF(hits_5 >= 2) AS repeat_5,
    SUM(hits_5) AS hits_5,
    COUNTIF(hits_6 > 0) AS users_6,
    COUNTIF(hits_6 = 1) AS once_6,
    COUNTIF(hits_6 >= 2) AS repeat_6,
    SUM(hits_6) AS hits_6
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'market_tab' AS step_id,
    'Marketplace bottom nav' AS step_label,
    'Marketplace_bottom_nav, marketplace_bottom_nav' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
```

```
  users_0 AS users,
  once_0 AS once_users,
  repeat_0 AS repeat_users,
  hits_0 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  2 AS step_order,
  'market_screen' AS step_id,
  'Marketplace screen' AS step_label,
  'marketplace_screen' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_1 AS users,
  once_1 AS once_users,
  repeat_1 AS repeat_users,
  hits_1 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  3 AS step_order,
  'market_item' AS step_id,
  'Listing tap (market_item_expolre)' AS step_label,
  'market_item_expolre' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_2 AS users,
  once_2 AS once_users,
  repeat_2 AS repeat_users,
  hits_2 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  4 AS step_order,
  'sale_details' AS step_id,
  'Sale details screen' AS step_label,
  'sale_Details_screen' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_3 AS users,
  once_3 AS once_users,
  repeat_3 AS repeat_users,
  hits_3 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  5 AS step_order,
  'contact' AS step_id,
  'Contact seller' AS step_label,
  'market_contact, market_contact_button' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_4 AS users,
  once_4 AS once_users,
  repeat_4 AS repeat_users,
  hits_4 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  6 AS step_order,
  'sell_cta' AS step_id,
  'Add for sale CTA' AS step_label,
  'add_for_sale_button_marketplace, add_for_sale_details_scr_btn' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_5 AS users,
  once_5 AS once_users,
```

```
    repeat_5 AS repeat_users,
    hits_5 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    7 AS step_order,
    'listing_created' AS step_id,
    'Listing published' AS step_label,
    'market_add, market_contact_Add_for_sale' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_6 AS users,
    once_6 AS once_users,
    repeat_6 AS repeat_users,
    hits_6 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Coinzy · marketplace

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('marketplace_bottom_nav', 'Marketplace_bottom_nav'))
AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('marketplace_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('market_item_expolre')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('sale_Details_screen')) AS hits_3,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('market_contact', 'market_contact_button')) AS
hits_4,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('add_for_sale_button_marketplace',
'add_for_sale_details_screen_button', 'add_for_sale_owned_item_button')) AS hits_5,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('market_add')) AS hits_6
```

```
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('marketplace_bottom_nav', 'Marketplace_bottom_nav',
'marketplace_screen', 'market_item_expolre', 'sale_Details_screen', 'market_contact', 'market_contact_button',
'add_for_sale_button_marketplace', 'add_for_sale_details_screen_button', 'add_for_sale_owned_item_button',
'market_add'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3,
    COUNTIF(hits_4 > 0) AS users_4,
    COUNTIF(hits_4 = 1) AS once_4,
    COUNTIF(hits_4 >= 2) AS repeat_4,
    SUM(hits_4) AS hits_4,
    COUNTIF(hits_5 > 0) AS users_5,
    COUNTIF(hits_5 = 1) AS once_5,
    COUNTIF(hits_5 >= 2) AS repeat_5,
    SUM(hits_5) AS hits_5,
    COUNTIF(hits_6 > 0) AS users_6,
    COUNTIF(hits_6 = 1) AS once_6,
    COUNTIF(hits_6 >= 2) AS repeat_6,
    SUM(hits_6) AS hits_6
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'market_tab' AS step_id,
    'Marketplace bottom nav' AS step_label,
    'marketplace_bottom_nav, Marketplace_bottom_nav' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'market_screen' AS step_id,
    'Marketplace screen' AS step_label,
    'marketplace_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
```

```
    3 AS step_order,
    'market_item' AS step_id,
    'Listing tap (market_item_expolre)' AS step_label,
    'market_item_expolre' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    4 AS step_order,
    'sale_details' AS step_id,
    'Sale details screen' AS step_label,
    'sale_Details_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_3 AS users,
    once_3 AS once_users,
    repeat_3 AS repeat_users,
    hits_3 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    5 AS step_order,
    'contact' AS step_id,
    'Contact seller' AS step_label,
    'market_contact, market_contact_button' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_4 AS users,
    once_4 AS once_users,
    repeat_4 AS repeat_users,
    hits_4 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    6 AS step_order,
    'sell_cta' AS step_id,
    'Add for sale CTA' AS step_label,
    'add_for_sale_button_marketplace, add_for_sale_details_screen_button, add_for_sale_owned_item_button' AS
event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_5 AS users,
    once_5 AS once_users,
    repeat_5 AS repeat_users,
    hits_5 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    7 AS step_order,
    'listing_created' AS step_id,
    'Listing published' AS step_label,
    'market_add' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_6 AS users,
    once_6 AS once_users,
    repeat_6 AS repeat_users,
    hits_6 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
```

```
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Banknote · feed

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('feed_bottom_nav')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Feed_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('feed_like', 'feed_comment')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('feed_add', 'feed_post')) AS hits_3
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('feed_bottom_nav', 'Feed_screen', 'feed_like', 'feed_comment',
'feed_add', 'feed_post'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3
  FROM per_user
  WHERE uid IS NOT NULL
```

```
),
steps AS (

  SELECT
    1 AS step_order,
    'feed_tab' AS step_id,
    'Feed bottom nav' AS step_label,
    'feed_bottom_nav' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'feed_screen' AS step_id,
    'Feed screen' AS step_label,
    'Feed_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'feed_engage' AS step_id,
    'Feed like / comment' AS step_label,
    'feed_like, feed_comment' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    4 AS step_order,
    'feed_post' AS step_id,
    'Feed create post' AS step_label,
    'feed_add, feed_post' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_3 AS users,
    once_3 AS once_users,
    repeat_3 AS repeat_users,
    hits_3 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
```

```
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Coinzy · feed

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('feed_bottom_nav')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Feed_screen')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('feed_like', 'feed_comment')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('feed_add')) AS hits_3
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('feed_bottom_nav', 'Feed_screen', 'feed_like', 'feed_comment',
'feed_add'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'feed_tab' AS step_id,
    'Feed bottom nav' AS step_label,
    'feed_bottom_nav' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
```

```
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'feed_screen' AS step_id,
    'Feed screen' AS step_label,
    'Feed_screen' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'feed_engage' AS step_id,
    'Feed like / comment' AS step_label,
    'feed_like, feed_comment' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    4 AS step_order,
    'feed_post' AS step_id,
    'Feed create post' AS step_label,
    'feed_add' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_3 AS users,
    once_3 AS once_users,
    repeat_3 AS repeat_users,
    hits_3 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
```

```
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Banknote · paywall

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Subs_page', 'Subs_page_discount',
'Subscription_screen', 'Subs_page_onboarding')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Subs_confirm')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('subs_cancel', 'Subs_cancel')) AS hits_2
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Subs_page', 'Subs_page_discount', 'Subscription_screen',
'Subs_page_onboarding', 'Subs_confirm', 'subs_cancel', 'Subs_cancel'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'paywall' AS step_id,
    'Paywall (Subs_page)' AS step_label,
    'Subs_page, Subs_page_discount, Subscription_screen, Subs_page_onboarding' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'confirm' AS step_id,
    'Purchase confirm (Subs_confirm)' AS step_label,
    'Subs_confirm' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
```

```
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'cancel' AS step_id,
    'Subs cancel' AS step_label,
    'subs_cancel, Subs_cancel' AS event_names,
    FALSE AS is_core,
    TRUE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Coinzy · paywall

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Subs_page', 'Subs_page_discount',
'Subscription_screen', 'Subs_page_onboarding')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('subs_pack', 'subs_pack_discount', 'subs_button')) AS
hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('subs_confirm', 'subs_confirm_discount',
'Subs_confirm', 'paid_purchase', 'trial_purchase')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('subs_cancel', 'Subs_cancel')) AS hits_3,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('subs_fail', 'Subs_fail')) AS hits_4
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
```

```
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('Subs_page', 'Subs_page_discount', 'Subscription_screen',
'Subs_page_onboarding', 'subs_pack', 'subs_pack_discount', 'subs_button', 'subs_confirm', 'subs_confirm_discount',
'Subs_confirm', 'paid_purchase', 'trial_purchase', 'subs_cancel', 'Subs_cancel', 'subs_fail', 'Subs_fail'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3,
    COUNTIF(hits_4 > 0) AS users_4,
    COUNTIF(hits_4 = 1) AS once_4,
    COUNTIF(hits_4 >= 2) AS repeat_4,
    SUM(hits_4) AS hits_4
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'paywall' AS step_id,
    'Paywall (Subs_page)' AS step_label,
    'Subs_page, Subs_page_discount, Subscription_screen, Subs_page_onboarding' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'pack' AS step_id,
    'Pack click' AS step_label,
    'subs_pack, subs_pack_discount, subs_button' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'confirm' AS step_id,
    'Purchase confirm (subs_confirm)' AS step_label,
    'subs_confirm, subs_confirm_discount, Subs_confirm, paid_purchase, trial_purchase' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
```

```
  UNION ALL

  SELECT
    4 AS step_order,
    'cancel' AS step_id,
    'Subs cancel' AS step_label,
    'subs_cancel, Subs_cancel' AS event_names,
    FALSE AS is_core,
    TRUE AS is_drop,
    users_3 AS users,
    once_3 AS once_users,
    repeat_3 AS repeat_users,
    hits_3 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    5 AS step_order,
    'fail' AS step_id,
    'Subs fail' AS step_label,
    'subs_fail, Subs_fail' AS event_names,
    FALSE AS is_core,
    TRUE AS is_drop,
    users_4 AS users,
    once_4 AS once_users,
    repeat_4 AS repeat_users,
    hits_4 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Coinzy · expert evaluation

```
WITH per_user AS (
  SELECT
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS uid,
```

```
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')) AS
dau_hits,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_evaluation_landing')) AS hits_0,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_evaluation_start')) AS hits_1,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_evaluations_list')) AS hits_2,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_evaluation_view_all')) AS hits_3,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_evaluation_item_click')) AS hits_4,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_upload_photos')) AS hits_5,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_upload_continue_with_credit',
'expert_upload_continue_payment')) AS hits_6,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_request_queued')) AS hits_7,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_evaluation_status')) AS hits_8,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_evaluation_report')) AS hits_9,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_rating_submitted')) AS hits_10,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_report_pdf_download')) AS hits_11,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_report_pdf_share')) AS hits_12,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_book_new_evaluation')) AS hits_13,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_book_new_evaluation_created')) AS hits_14,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_outbox_retry')) AS hits_15,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_outbox_retry_after_credits')) AS hits_16,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_request_retry_started')) AS hits_17,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_refund_requested')) AS hits_18,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_report_pdf_failed')) AS hits_19,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_evaluation_buy_credits')) AS hits_20,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_buy_credits_continue_payment',
'expert_status_buy_credits_continue_payment')) AS hits_21,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_token_purchase_received')) AS hits_22,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_token_purchase_consumed')) AS hits_23,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_token_purchase_pending')) AS hits_24,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_token_purchase_cancelled')) AS hits_25,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_token_purchase_failed')) AS hits_26,
    COUNTIF(REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_token_verification_failed')) AS hits_27
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND (REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open') OR
REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('expert_evaluation_landing', 'expert_evaluation_start',
'expert_evaluations_list', 'expert_evaluation_view_all', 'expert_evaluation_item_click', 'expert_upload_photos',
'expert_upload_continue_with_credit', 'expert_upload_continue_payment', 'expert_request_queued',
'expert_evaluation_status', 'expert_evaluation_report', 'expert_rating_submitted', 'expert_report_pdf_download',
'expert_report_pdf_share', 'expert_book_new_evaluation', 'expert_book_new_evaluation_created', 'expert_outbox_retry',
'expert_outbox_retry_after_credits', 'expert_request_retry_started', 'expert_refund_requested',
'expert_report_pdf_failed', 'expert_evaluation_buy_credits', 'expert_buy_credits_continue_payment',
'expert_status_buy_credits_continue_payment', 'expert_token_purchase_received', 'expert_token_purchase_consumed',
'expert_token_purchase_pending', 'expert_token_purchase_cancelled', 'expert_token_purchase_failed',
'expert_token_verification_failed'))
  GROUP BY uid
),
agg AS (
  SELECT
    COUNTIF(dau_hits > 0) AS dau,
    COUNTIF(hits_0 > 0) AS users_0,
    COUNTIF(hits_0 = 1) AS once_0,
    COUNTIF(hits_0 >= 2) AS repeat_0,
    SUM(hits_0) AS hits_0,
    COUNTIF(hits_1 > 0) AS users_1,
    COUNTIF(hits_1 = 1) AS once_1,
    COUNTIF(hits_1 >= 2) AS repeat_1,
    SUM(hits_1) AS hits_1,
    COUNTIF(hits_2 > 0) AS users_2,
    COUNTIF(hits_2 = 1) AS once_2,
    COUNTIF(hits_2 >= 2) AS repeat_2,
    SUM(hits_2) AS hits_2,
    COUNTIF(hits_3 > 0) AS users_3,
    COUNTIF(hits_3 = 1) AS once_3,
    COUNTIF(hits_3 >= 2) AS repeat_3,
    SUM(hits_3) AS hits_3,
    COUNTIF(hits_4 > 0) AS users_4,
    COUNTIF(hits_4 = 1) AS once_4,
    COUNTIF(hits_4 >= 2) AS repeat_4,
    SUM(hits_4) AS hits_4,
    COUNTIF(hits_5 > 0) AS users_5,
    COUNTIF(hits_5 = 1) AS once_5,
    COUNTIF(hits_5 >= 2) AS repeat_5,
    SUM(hits_5) AS hits_5,
    COUNTIF(hits_6 > 0) AS users_6,
    COUNTIF(hits_6 = 1) AS once_6,
```

```
    COUNTIF(hits_6 >= 2) AS repeat_6,
    SUM(hits_6) AS hits_6,
    COUNTIF(hits_7 > 0) AS users_7,
    COUNTIF(hits_7 = 1) AS once_7,
    COUNTIF(hits_7 >= 2) AS repeat_7,
    SUM(hits_7) AS hits_7,
    COUNTIF(hits_8 > 0) AS users_8,
    COUNTIF(hits_8 = 1) AS once_8,
    COUNTIF(hits_8 >= 2) AS repeat_8,
    SUM(hits_8) AS hits_8,
    COUNTIF(hits_9 > 0) AS users_9,
    COUNTIF(hits_9 = 1) AS once_9,
    COUNTIF(hits_9 >= 2) AS repeat_9,
    SUM(hits_9) AS hits_9,
    COUNTIF(hits_10 > 0) AS users_10,
    COUNTIF(hits_10 = 1) AS once_10,
    COUNTIF(hits_10 >= 2) AS repeat_10,
    SUM(hits_10) AS hits_10,
    COUNTIF(hits_11 > 0) AS users_11,
    COUNTIF(hits_11 = 1) AS once_11,
    COUNTIF(hits_11 >= 2) AS repeat_11,
    SUM(hits_11) AS hits_11,
    COUNTIF(hits_12 > 0) AS users_12,
    COUNTIF(hits_12 = 1) AS once_12,
    COUNTIF(hits_12 >= 2) AS repeat_12,
    SUM(hits_12) AS hits_12,
    COUNTIF(hits_13 > 0) AS users_13,
    COUNTIF(hits_13 = 1) AS once_13,
    COUNTIF(hits_13 >= 2) AS repeat_13,
    SUM(hits_13) AS hits_13,
    COUNTIF(hits_14 > 0) AS users_14,
    COUNTIF(hits_14 = 1) AS once_14,
    COUNTIF(hits_14 >= 2) AS repeat_14,
    SUM(hits_14) AS hits_14,
    COUNTIF(hits_15 > 0) AS users_15,
    COUNTIF(hits_15 = 1) AS once_15,
    COUNTIF(hits_15 >= 2) AS repeat_15,
    SUM(hits_15) AS hits_15,
    COUNTIF(hits_16 > 0) AS users_16,
    COUNTIF(hits_16 = 1) AS once_16,
    COUNTIF(hits_16 >= 2) AS repeat_16,
    SUM(hits_16) AS hits_16,
    COUNTIF(hits_17 > 0) AS users_17,
    COUNTIF(hits_17 = 1) AS once_17,
    COUNTIF(hits_17 >= 2) AS repeat_17,
    SUM(hits_17) AS hits_17,
    COUNTIF(hits_18 > 0) AS users_18,
    COUNTIF(hits_18 = 1) AS once_18,
    COUNTIF(hits_18 >= 2) AS repeat_18,
    SUM(hits_18) AS hits_18,
    COUNTIF(hits_19 > 0) AS users_19,
    COUNTIF(hits_19 = 1) AS once_19,
    COUNTIF(hits_19 >= 2) AS repeat_19,
    SUM(hits_19) AS hits_19,
    COUNTIF(hits_20 > 0) AS users_20,
    COUNTIF(hits_20 = 1) AS once_20,
    COUNTIF(hits_20 >= 2) AS repeat_20,
    SUM(hits_20) AS hits_20,
    COUNTIF(hits_21 > 0) AS users_21,
    COUNTIF(hits_21 = 1) AS once_21,
    COUNTIF(hits_21 >= 2) AS repeat_21,
    SUM(hits_21) AS hits_21,
    COUNTIF(hits_22 > 0) AS users_22,
    COUNTIF(hits_22 = 1) AS once_22,
    COUNTIF(hits_22 >= 2) AS repeat_22,
    SUM(hits_22) AS hits_22,
    COUNTIF(hits_23 > 0) AS users_23,
    COUNTIF(hits_23 = 1) AS once_23,
    COUNTIF(hits_23 >= 2) AS repeat_23,
    SUM(hits_23) AS hits_23,
    COUNTIF(hits_24 > 0) AS users_24,
    COUNTIF(hits_24 = 1) AS once_24,
    COUNTIF(hits_24 >= 2) AS repeat_24,
    SUM(hits_24) AS hits_24,
    COUNTIF(hits_25 > 0) AS users_25,
    COUNTIF(hits_25 = 1) AS once_25,
    COUNTIF(hits_25 >= 2) AS repeat_25,
```

```
    SUM(hits_25) AS hits_25,
    COUNTIF(hits_26 > 0) AS users_26,
    COUNTIF(hits_26 = 1) AS once_26,
    COUNTIF(hits_26 >= 2) AS repeat_26,
    SUM(hits_26) AS hits_26,
    COUNTIF(hits_27 > 0) AS users_27,
    COUNTIF(hits_27 = 1) AS once_27,
    COUNTIF(hits_27 >= 2) AS repeat_27,
    SUM(hits_27) AS hits_27
  FROM per_user
  WHERE uid IS NOT NULL
),
steps AS (

  SELECT
    1 AS step_order,
    'landing' AS step_id,
    'Expert landing' AS step_label,
    'expert_evaluation_landing' AS event_names,
    TRUE AS is_core,
    FALSE AS is_drop,
    users_0 AS users,
    once_0 AS once_users,
    repeat_0 AS repeat_users,
    hits_0 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    2 AS step_order,
    'start' AS step_id,
    'Start evaluation' AS step_label,
    'expert_evaluation_start' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_1 AS users,
    once_1 AS once_users,
    repeat_1 AS repeat_users,
    hits_1 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    3 AS step_order,
    'list' AS step_id,
    'Evaluations list' AS step_label,
    'expert_evaluations_list' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_2 AS users,
    once_2 AS once_users,
    repeat_2 AS repeat_users,
    hits_2 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    4 AS step_order,
    'view_all' AS step_id,
    'View all evaluations' AS step_label,
    'expert_evaluation_view_all' AS event_names,
    FALSE AS is_core,
    FALSE AS is_drop,
    users_3 AS users,
    once_3 AS once_users,
    repeat_3 AS repeat_users,
    hits_3 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    5 AS step_order,
    'item_click' AS step_id,
```

```
  'Open an evaluation' AS step_label,
  'expert_evaluation_item_click' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_4 AS users,
  once_4 AS once_users,
  repeat_4 AS repeat_users,
  hits_4 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  6 AS step_order,
  'upload' AS step_id,
  'Upload photos' AS step_label,
  'expert_upload_photos' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_5 AS users,
  once_5 AS once_users,
  repeat_5 AS repeat_users,
  hits_5 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  7 AS step_order,
  'continue' AS step_id,
  'Continue (credit or pay)' AS step_label,
  'expert_upload_continue_with_credit, expert_upload_continue_payment' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_6 AS users,
  once_6 AS once_users,
  repeat_6 AS repeat_users,
  hits_6 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  8 AS step_order,
  'queued' AS step_id,
  'Request queued' AS step_label,
  'expert_request_queued' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_7 AS users,
  once_7 AS once_users,
  repeat_7 AS repeat_users,
  hits_7 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  9 AS step_order,
  'status' AS step_id,
  'Evaluation status' AS step_label,
  'expert_evaluation_status' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_8 AS users,
  once_8 AS once_users,
  repeat_8 AS repeat_users,
  hits_8 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  10 AS step_order,
  'report' AS step_id,
  'Expert report' AS step_label,
  'expert_evaluation_report' AS event_names,
```

```
  TRUE AS is_core,
  FALSE AS is_drop,
  users_9 AS users,
  once_9 AS once_users,
  repeat_9 AS repeat_users,
  hits_9 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  11 AS step_order,
  'rating' AS step_id,
  'Rating submitted' AS step_label,
  'expert_rating_submitted' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_10 AS users,
  once_10 AS once_users,
  repeat_10 AS repeat_users,
  hits_10 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  12 AS step_order,
  'pdf_download' AS step_id,
  'Download report PDF' AS step_label,
  'expert_report_pdf_download' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_11 AS users,
  once_11 AS once_users,
  repeat_11 AS repeat_users,
  hits_11 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  13 AS step_order,
  'pdf_share' AS step_id,
  'Share report PDF' AS step_label,
  'expert_report_pdf_share' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_12 AS users,
  once_12 AS once_users,
  repeat_12 AS repeat_users,
  hits_12 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  14 AS step_order,
  'book_new' AS step_id,
  'Book new evaluation' AS step_label,
  'expert_book_new_evaluation' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_13 AS users,
  once_13 AS once_users,
  repeat_13 AS repeat_users,
  hits_13 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  15 AS step_order,
  'book_created' AS step_id,
  'New evaluation created' AS step_label,
  'expert_book_new_evaluation_created' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
```

```
  users_14 AS users,
  once_14 AS once_users,
  repeat_14 AS repeat_users,
  hits_14 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  16 AS step_order,
  'retry_outbox' AS step_id,
  'Outbox retry' AS step_label,
  'expert_outbox_retry' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_15 AS users,
  once_15 AS once_users,
  repeat_15 AS repeat_users,
  hits_15 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  17 AS step_order,
  'retry_after_credits' AS step_id,
  'Retry after credits' AS step_label,
  'expert_outbox_retry_after_credits' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_16 AS users,
  once_16 AS once_users,
  repeat_16 AS repeat_users,
  hits_16 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  18 AS step_order,
  'retry_started' AS step_id,
  'Request retry started' AS step_label,
  'expert_request_retry_started' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_17 AS users,
  once_17 AS once_users,
  repeat_17 AS repeat_users,
  hits_17 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  19 AS step_order,
  'refund' AS step_id,
  'Refund requested' AS step_label,
  'expert_refund_requested' AS event_names,
  FALSE AS is_core,
  TRUE AS is_drop,
  users_18 AS users,
  once_18 AS once_users,
  repeat_18 AS repeat_users,
  hits_18 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  20 AS step_order,
  'pdf_failed' AS step_id,
  'Report PDF failed' AS step_label,
  'expert_report_pdf_failed' AS event_names,
  FALSE AS is_core,
  TRUE AS is_drop,
  users_19 AS users,
  once_19 AS once_users,
```

```
  repeat_19 AS repeat_users,
  hits_19 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  21 AS step_order,
  'buy_credits' AS step_id,
  'Buy expert credits' AS step_label,
  'expert_evaluation_buy_credits' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_20 AS users,
  once_20 AS once_users,
  repeat_20 AS repeat_users,
  hits_20 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  22 AS step_order,
  'credits_continue' AS step_id,
  'Credits → continue payment' AS step_label,
  'expert_buy_credits_continue_payment, expert_status_buy_credits_continue_payment' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_21 AS users,
  once_21 AS once_users,
  repeat_21 AS repeat_users,
  hits_21 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  23 AS step_order,
  'token_received' AS step_id,
  'Token purchase received' AS step_label,
  'expert_token_purchase_received' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_22 AS users,
  once_22 AS once_users,
  repeat_22 AS repeat_users,
  hits_22 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  24 AS step_order,
  'token_consumed' AS step_id,
  'Token consumed' AS step_label,
  'expert_token_purchase_consumed' AS event_names,
  TRUE AS is_core,
  FALSE AS is_drop,
  users_23 AS users,
  once_23 AS once_users,
  repeat_23 AS repeat_users,
  hits_23 AS hits,
  dau
FROM agg
UNION ALL

SELECT
  25 AS step_order,
  'token_pending' AS step_id,
  'Token purchase pending' AS step_label,
  'expert_token_purchase_pending' AS event_names,
  FALSE AS is_core,
  FALSE AS is_drop,
  users_24 AS users,
  once_24 AS once_users,
  repeat_24 AS repeat_users,
  hits_24 AS hits,
```

```
    dau
  FROM agg
  UNION ALL

  SELECT
    26 AS step_order,
    'token_cancelled' AS step_id,
    'Token purchase cancelled' AS step_label,
    'expert_token_purchase_cancelled' AS event_names,
    FALSE AS is_core,
    TRUE AS is_drop,
    users_25 AS users,
    once_25 AS once_users,
    repeat_25 AS repeat_users,
    hits_25 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    27 AS step_order,
    'token_failed' AS step_id,
    'Token purchase failed' AS step_label,
    'expert_token_purchase_failed' AS event_names,
    FALSE AS is_core,
    TRUE AS is_drop,
    users_26 AS users,
    once_26 AS once_users,
    repeat_26 AS repeat_users,
    hits_26 AS hits,
    dau
  FROM agg
  UNION ALL

  SELECT
    28 AS step_order,
    'token_verify_fail' AS step_id,
    'Token verification failed' AS step_label,
    'expert_token_verification_failed' AS event_names,
    FALSE AS is_core,
    TRUE AS is_drop,
    users_27 AS users,
    once_27 AS once_users,
    repeat_27 AS repeat_users,
    hits_27 AS hits,
    dau
  FROM agg
),
core_chain AS (
  SELECT
    step_id,
    users,
    LAG(users) OVER (ORDER BY step_order) AS prev_users
  FROM steps
  WHERE is_core AND NOT is_drop
)
SELECT
  s.step_order,
  s.step_id,
  s.step_label,
  s.event_names,
  s.is_core,
  s.is_drop,
  s.users,
  s.once_users,
  s.repeat_users,
  s.hits,
  s.dau,
  SAFE_DIVIDE(s.users, s.dau) AS pct_of_dau,
  SAFE_DIVIDE(s.hits, NULLIF(s.users, 0)) AS hits_per_user,
  SAFE_DIVIDE(s.repeat_users, NULLIF(s.users, 0)) AS repeat_share,
  c.prev_users,
  SAFE_DIVIDE(s.users, c.prev_users) AS pct_of_previous,
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE GREATEST(0, c.prev_users - s.users)
  END AS drop_off_users,
```

```
  CASE
    WHEN c.prev_users IS NULL THEN NULL
    ELSE 1 - SAFE_DIVIDE(s.users, c.prev_users)
  END AS drop_off_rate
FROM steps s
LEFT JOIN core_chain c ON c.step_id = s.step_id
ORDER BY s.step_order
```

Event inventory

> Built in funnel-registry.js. List prefers analytics_summary.top_events; click-through is always raw.

List events (raw fallback)

```
SELECT
  REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name,
  COUNT(*) AS hits,
  COUNT(DISTINCT COALESCE(
    IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous','null','undefined','(not
set)','(anonymous)'), NULL, TRIM(user_id)),
    user_pseudo_id
  )) AS unique_users,
  MIN(PARSE_DATE('%Y%m%d', event_date)) AS first_seen,
  MAX(PARSE_DATE('%Y%m%d', event_date)) AS last_seen
FROM `{PROJECT}.{DATASET}.events_*`
WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', {{start_date}}) AND FORMAT_DATE('%Y%m%d', {{end_date}})
  AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\\d{8}$')
GROUP BY event_name
ORDER BY hits DESC
LIMIT 500
```

Explorer — Unique vs repeat

Query

`sql/dashboard/raw/19_user_mix.sql`

```
-- ================================================================================
-- Unique vs new vs returning vs one-open vs repeat (same DAU events)
-- Qualifying: session_start, App_open, first_open (suffix-stripped)
-- Daily rows + range_* totals repeated on each row for KPI cards
-- ================================================================================

WITH base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(IF(user_id IS NULL OR TRIM(user_id) = '' OR LOWER(TRIM(user_id)) IN ('anonymous', 'null', 'undefined',
'(not set)', '(anonymous)'), NULL, TRIM(user_id)), NULL, user_pseudo_id) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`
  WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', {{start_date}})
                          AND FORMAT_DATE('%Y%m%d', {{end_date}})
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN ('session_start', 'App_open', 'first_open')
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
),
per_user_day AS (
  SELECT
    event_date,
    resolved_user_id,
    COUNT(*) AS opens,
    COUNTIF(event_name_base = 'first_open') AS first_open_hits
  FROM base
  WHERE resolved_user_id IS NOT NULL
  GROUP BY event_date, resolved_user_id
),
daily AS (
  SELECT
```

```
    event_date,
    COUNT(*) AS unique_users,
    COUNTIF(first_open_hits > 0) AS new_users,
    COUNTIF(first_open_hits = 0) AS returning_users,
    COUNTIF(opens = 1) AS one_time_users,
    COUNTIF(opens >= 2) AS repeat_users,
    SUM(opens) AS opens,
    SAFE_DIVIDE(SUM(opens), COUNT(*)) AS opens_per_user,
    SAFE_DIVIDE(COUNTIF(first_open_hits = 0), COUNT(*)) AS returning_share,
    SAFE_DIVIDE(COUNTIF(opens >= 2), COUNT(*)) AS repeat_share
  FROM per_user_day
  GROUP BY event_date
),
per_user_range AS (
  SELECT
    resolved_user_id,
    SUM(opens) AS opens,
    SUM(first_open_hits) AS first_open_hits,
    COUNT(*) AS active_days
  FROM per_user_day
  GROUP BY resolved_user_id
),
rng AS (
  SELECT
    COUNT(*) AS range_unique_users,
    COUNTIF(first_open_hits > 0) AS range_new_users,
    COUNTIF(first_open_hits = 0) AS range_returning_users,
    COUNTIF(active_days = 1) AS range_one_day_users,
    COUNTIF(active_days >= 2) AS range_multi_day_users,
    SUM(opens) AS range_opens,
    SAFE_DIVIDE(SUM(opens), COUNT(*)) AS range_opens_per_user,
    SAFE_DIVIDE(COUNTIF(first_open_hits = 0), COUNT(*)) AS range_returning_share,
    SAFE_DIVIDE(COUNTIF(active_days >= 2), COUNT(*)) AS range_multi_day_share
  FROM per_user_range
)
SELECT
  d.event_date,
  d.unique_users,
  d.new_users,
  d.returning_users,
  d.one_time_users,
  d.repeat_users,
  d.opens,
  d.opens_per_user,
  d.returning_share,
  d.repeat_share,
  r.range_unique_users,
  r.range_new_users,
  r.range_returning_users,
  r.range_one_day_users,
  r.range_multi_day_users,
  r.range_opens,
  r.range_opens_per_user,
  r.range_returning_share,
  r.range_multi_day_share
FROM daily d
CROSS JOIN rng r
ORDER BY d.event_date;
```

Explorer — Monthly Active Users

Query

`sql/dashboard/raw/02_mau.sql`

```
-- ================================================================================
-- Raw-events MAU (no views required — works with Data Viewer only)
-- ================================================================================

SELECT
  DATE_TRUNC(PARSE_DATE('%Y%m%d', event_date), MONTH) AS activity_month,
  COUNT(DISTINCT COALESCE(
    user_id,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
```

```
    user_pseudo_id
  )) AS mau
FROM `{PROJECT}.{DATASET}.events_*`
WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', DATE_TRUNC({{start_date}}, MONTH))
                        AND FORMAT_DATE('%Y%m%d', {{end_date}})
  AND _TABLE_SUFFIX NOT LIKE 'intraday_%'
  [[AND event_country = {{country}}]]
  [[AND event_platform = {{platform}}]]
GROUP BY activity_month
ORDER BY activity_month;
```

Explorer — New Users

Query

sql/dashboard/raw/03_new_users.sql

```
-- ================================================================================
-- Raw-events new users (first_open)
-- ================================================================================

SELECT
  PARSE_DATE('%Y%m%d', event_date) AS cohort_date,
  COUNT(DISTINCT COALESCE(
    user_id,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
    user_pseudo_id
  )) AS new_users
FROM `{PROJECT}.{DATASET}.events_*`
WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', {{start_date}})
                        AND FORMAT_DATE('%Y%m%d', {{end_date}})
  AND _TABLE_SUFFIX NOT LIKE 'intraday_%'
  AND (
    event_name IN ('first_open', 'first_open_android', 'first_open_ios')
    OR REGEXP_CONTAINS(event_name, r'^first_open')
  )
  [[AND event_country = {{country}}]]
  [[AND event_platform = {{platform}}]]
GROUP BY cohort_date
ORDER BY cohort_date;
```

Explorer — Installs + time used

Query

sql/dashboard/raw/20_install_day_usage.sql

```
-- ================================================================================
-- Install day usage
-- Installs = distinct devices (user_pseudo_id) with first_open that calendar day.
-- Went in  = those devices with >= 10 seconds in the app that same day
--            (Firebase user_engagement.engagement_time_msec, else session_length_seconds).
-- Time     = seconds in the app on the install day (among those who went in).
-- Do not join on user_id — same reason as same-day first ID.
-- ================================================================================

WITH bounds AS (
  SELECT
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    user_pseudo_id AS device_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base,
    (SELECT ep.value.int_value FROM UNNEST(event_params) ep WHERE ep.key = 'engagement_time_msec')
      AS engagement_time_msec,
    (SELECT ep.value.int_value FROM UNNEST(event_params) ep WHERE ep.key = 'session_length_seconds')
      AS session_length_seconds
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
```

```
  WHERE _TABLE_SUFFIX BETWEEN start_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND user_pseudo_id IS NOT NULL
    AND TRIM(user_pseudo_id) != ''
    AND (
      STARTS_WITH(REGEXP_REPLACE(event_name, r'_(android|ios)$', ''), 'first_open')
      OR REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN (
        'session_start', 'App_open', 'user_engagement'
      )
    )
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
),
per_device_day AS (
  SELECT
    device_id,
    event_date,
    COUNTIF(
      event_name_base = 'first_open'
      OR STARTS_WITH(event_name_base, 'first_open')
    ) > 0 AS is_install,
    IFNULL(SUM(IF(event_name_base = 'user_engagement', engagement_time_msec, 0)), 0) / 1000.0
      AS engagement_seconds,
    IFNULL(MAX(session_length_seconds), 0) AS session_length_seconds
  FROM base
  GROUP BY device_id, event_date
),
installers AS (
  SELECT
    device_id,
    event_date,
    GREATEST(engagement_seconds, session_length_seconds) AS time_seconds
  FROM per_device_day
  WHERE is_install
)
SELECT
  event_date,
  COUNT(*) AS installs,
  COUNTIF(time_seconds >= 10) AS went_in,
  SAFE_DIVIDE(COUNTIF(time_seconds >= 10), COUNT(*)) AS went_in_rate,
  SUM(IF(time_seconds >= 10, time_seconds, 0)) AS total_seconds_went_in,
  AVG(IF(time_seconds >= 10, time_seconds, NULL)) AS avg_seconds,
  APPROX_QUANTILES(
    IF(time_seconds >= 10, time_seconds, NULL),
    100
  )[OFFSET(50)] AS median_seconds
FROM installers
GROUP BY event_date
ORDER BY event_date;
```

Explorer — Scan limits

Query

`sql/dashboard/raw/21_scan_limits.sql`

```
-- ================================================================================
-- Scan limits: free vs subscribed, success quota vs unsuccessful quota
--
-- Free success:   free_scan_success_quota_exhausted, free_scan_limit_exceeded,
--                 free_scan_blocked, Identified_limit_reached,
--                 identiifcation_limit_exceeded, scan_quota_exhausted
-- Free fail:      free_scan_fail_quota_exhausted,
--                 Identification_unsuccessful_limit_reached
-- Subscribed:     same limit events, but the user had a purchase on or before
--                 that day (no separate Pro limit event exists in Firebase).
-- Identity: NULLIF(TRIM(user_id),'') then user_pseudo_id (empty user_id ignored).
-- ================================================================================

WITH bounds AS (
  SELECT
    {{start_date}} AS start_d,
    {{end_date}} AS end_d,
    DATE_SUB({{start_date}}, INTERVAL 180 DAY) AS paid_from,
```

```
    FORMAT_DATE('%Y%m%d', DATE_SUB({{start_date}}, INTERVAL 180 DAY)) AS paid_from_s,
    FORMAT_DATE('%Y%m%d', {{start_date}}) AS start_s,
    FORMAT_DATE('%Y%m%d', {{end_date}}) AS end_s
),
base AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    COALESCE(NULLIF(TRIM(user_id), ''), user_pseudo_id) AS uid,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN paid_from_s AND end_s
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND COALESCE(NULLIF(TRIM(user_id), ''), user_pseudo_id) IS NOT NULL
    AND (
      REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN (
        'Subs_confirm', 'subs_confirm', 'subs_confirm_discount',
        'paid_purchase', 'trial_purchase', 'in_app_purchase', 'life_time_access'
      )
      OR (
        _TABLE_SUFFIX >= start_s
        AND REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN (
          'Identified_limit_reached', 'identified_limit_reached',
          'identiifcation_limit_exceeded', 'identification_limit_exceeded',
          'scan_quota_exhausted', 'Scan_quota_exhausted',
          'limit_exceeded', 'Limit_exceeded',
          'free_scan_limit_exceeded', 'free_scan_blocked',
          'free_scan_success_quota_exhausted', 'free_scan_fail_quota_exhausted',
          'Identification_unsuccessful_limit_reached',
          'identification_done_success', 'Identification_done_success',
          'identification_done_failure', 'Identification_done_failure',
          'Identification_failed', 'Identification_unsuccessful',
          'Identification_done'
        )
      )
    )
    [[AND event_country = {{country}}]]
    [[AND event_platform = {{platform}}]]
),
paid AS (
  SELECT
    uid,
    MIN(event_date) AS paid_on
  FROM base
  WHERE event_name_base IN (
    'Subs_confirm', 'subs_confirm', 'subs_confirm_discount',
    'paid_purchase', 'trial_purchase', 'in_app_purchase', 'life_time_access'
  )
  GROUP BY uid
),
daily AS (
  SELECT
    b.event_date,
    b.uid,
    MAX(IF(p.uid IS NOT NULL AND p.paid_on <= b.event_date, 1, 0)) AS is_subscribed,
    MAX(IF(b.event_name_base IN (
      'identification_done_success', 'Identification_done_success',
      'identification_done_failure', 'Identification_done_failure',
      'Identification_failed', 'Identification_unsuccessful',
      'Identification_done'
    ), 1, 0)) AS scanned,
    MAX(IF(b.event_name_base IN (
      'free_scan_success_quota_exhausted',
      'free_scan_limit_exceeded',
      'free_scan_blocked',
      'Identified_limit_reached', 'identified_limit_reached',
      'identiifcation_limit_exceeded', 'identification_limit_exceeded',
      'scan_quota_exhausted', 'Scan_quota_exhausted',
      'limit_exceeded', 'Limit_exceeded'
    ), 1, 0)) AS hit_success_limit,
    MAX(IF(b.event_name_base IN (
      'free_scan_fail_quota_exhausted',
      'Identification_unsuccessful_limit_reached'
    ), 1, 0)) AS hit_fail_limit
  FROM base b
  CROSS JOIN bounds
  LEFT JOIN paid p ON b.uid = p.uid
  WHERE b.event_date BETWEEN bounds.start_d AND bounds.end_d
```

```
  GROUP BY b.event_date, b.uid
)
SELECT
  event_date,
  COUNTIF(scanned = 1 AND is_subscribed = 0) AS free_scanners,
  COUNTIF(scanned = 1 AND is_subscribed = 1) AS subscribed_scanners,
  COUNTIF(hit_success_limit = 1 AND is_subscribed = 0) AS free_success_limit_users,
  COUNTIF(hit_fail_limit = 1 AND is_subscribed = 0) AS free_fail_limit_users,
  COUNTIF(hit_success_limit = 1 AND is_subscribed = 1) AS subscribed_success_limit_users,
  COUNTIF(hit_fail_limit = 1 AND is_subscribed = 1) AS subscribed_fail_limit_users,
  SAFE_DIVIDE(
    COUNTIF(hit_success_limit = 1 AND is_subscribed = 0),
    COUNTIF(scanned = 1 AND is_subscribed = 0)
  ) AS free_success_limit_rate,
  SAFE_DIVIDE(
    COUNTIF(hit_fail_limit = 1 AND is_subscribed = 0),
    COUNTIF(scanned = 1 AND is_subscribed = 0)
  ) AS free_fail_limit_rate,
  SAFE_DIVIDE(
    COUNTIF(hit_success_limit = 1 AND is_subscribed = 1),
    COUNTIF(scanned = 1 AND is_subscribed = 1)
  ) AS subscribed_success_limit_rate,
  SAFE_DIVIDE(
    COUNTIF(hit_fail_limit = 1 AND is_subscribed = 1),
    COUNTIF(scanned = 1 AND is_subscribed = 1)
  ) AS subscribed_fail_limit_rate
FROM daily
GROUP BY event_date
ORDER BY event_date;
```

Explorer — Top Countries

Query

`sql/dashboard/raw/04_countries.sql`

```
-- ============================================================================
-- Raw-events top countries
-- ============================================================================

SELECT
  COALESCE(
    NULLIF((SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'country'), ''),
    NULLIF(geo.country, ''),
    'Unknown'
  ) AS country,
  COUNT(DISTINCT COALESCE(
    user_id,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
    user_pseudo_id
  )) AS total_new_users
FROM `{PROJECT}.{DATASET}.events_*`
WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', {{start_date}})
                        AND FORMAT_DATE('%Y%m%d', {{end_date}})
  AND _TABLE_SUFFIX NOT LIKE 'intraday_%'
  [[AND event_platform = {{platform}}]]
GROUP BY country
HAVING country != 'Unknown'
ORDER BY total_new_users DESC
LIMIT 40;
```

Explorer — Platform

Query

`sql/dashboard/raw/08_platform.sql`

```
-- ============================================================================
-- Raw-events platform split
-- ============================================================================
```

```
SELECT
  LOWER(COALESCE(device.operating_system, platform, 'unknown')) AS platform,
  COUNT(DISTINCT COALESCE(
    user_id,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'user_id'),
    user_pseudo_id
  )) AS unique_users
FROM `{PROJECT}.{DATASET}.events_*`
WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', {{start_date}})
                        AND FORMAT_DATE('%Y%m%d', {{end_date}})
  AND _TABLE_SUFFIX NOT LIKE 'intraday_%'
  [[AND event_country = {{country}}]]
GROUP BY platform
ORDER BY unique_users DESC;
```

Explorer — Top Events

Query

`sql/dashboard/raw/07_top_events.sql`

```
-- ============================================================================
-- Raw-events top events
-- ============================================================================

SELECT
  REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base,
  COUNT(*) AS event_count
FROM `{PROJECT}.{DATASET}.events_*`
WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', {{start_date}})
                        AND FORMAT_DATE('%Y%m%d', {{end_date}})
  AND _TABLE_SUFFIX NOT LIKE 'intraday_%'
  [[AND event_country = {{country}}]]
  [[AND event_platform = {{platform}}]]
GROUP BY event_name_base
ORDER BY event_count DESC
LIMIT 25;
```

Explorer — Cohort LTV (emergency BigQuery only)

> Normal path is Mongo cohort_ltv (no BigQuery on click). This SQL is used only for LTV_FORCE_RAW / empty Mongo fallback.

Query

`sql/dashboard/raw/10_cohort_ltv.sql`

```
-- ============================================================================
-- Cohort LTV — primary dashboard query on existing Firebase export (events_*)
-- Not using analytics_summary. Grain: cohort_date × country × install_channel
--
-- Cohort: first first_open per user (install date + country + first-touch channel frozen)
-- Revenue: in_app_purchase / purchase monetary value only (event_value_in_usd or
--          event_params.value). Refunds subtract. Custom Subs_confirm is NOT revenue.
-- Maturity: LTV-N and revenue_N are NULL until DATE_DIFF(today, cohort_date) >= N.
-- Identity: COALESCE(user_id, user_pseudo_id) — do not UNNEST event_params.user_id
--           on this interactive path (cost standard).
--
-- Channel mapping verified on Coinzy + Banknote first_open (Aug 2026):
--   google-play / organic, google / organic → Organic
--   google / cpc                            → Paid
--   (direct) / (none)                       → Direct
-- collected_traffic_source is empty on first_open; used only as fallback.
-- ============================================================================

WITH bounds AS (
  SELECT
    {{start_date}} AS cohort_start,
    {{end_date}} AS cohort_end,
```

```
    LEAST(DATE_ADD({{end_date}}, INTERVAL 180 DAY), CURRENT_DATE()) AS purchase_end
),

events_in_window AS (
  SELECT
    PARSE_DATE('%Y%m%d', event_date) AS event_date,
    TIMESTAMP_MICROS(event_timestamp) AS event_timestamp,
    COALESCE(user_id, user_pseudo_id) AS resolved_user_id,
    REGEXP_REPLACE(event_name, r'_(android|ios)$', '') AS event_name_base,
    event_name,
    geo.country AS geo_country,
    device.operating_system AS device_os,
    platform,
    traffic_source.source AS ts_source,
    traffic_source.medium AS ts_medium,
    collected_traffic_source.manual_source AS cts_source,
    collected_traffic_source.manual_medium AS cts_medium,
    collected_traffic_source.gclid AS cts_gclid,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'country') AS param_country,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'platform') AS param_platform,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'utm_source') AS utm_source,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key = 'utm_medium') AS utm_medium,
    COALESCE(
      event_value_in_usd,
      (SELECT ep.value.double_value FROM UNNEST(event_params) ep WHERE ep.key = 'value'),
      (SELECT ep.value.float_value FROM UNNEST(event_params) ep WHERE ep.key = 'value'),
      SAFE_CAST((SELECT ep.value.int_value FROM UNNEST(event_params) ep WHERE ep.key = 'value') AS FLOAT64)
    ) AS amount_usd
  FROM `{PROJECT}.{DATASET}.events_*`, bounds
  WHERE _TABLE_SUFFIX BETWEEN FORMAT_DATE('%Y%m%d', cohort_start)
                          AND FORMAT_DATE('%Y%m%d', purchase_end)
    AND _TABLE_SUFFIX NOT LIKE 'intraday_%'
    AND REGEXP_CONTAINS(_TABLE_SUFFIX, r'^\d{8}$')
    AND REGEXP_REPLACE(event_name, r'_(android|ios)$', '') IN (
      'first_open', 'in_app_purchase', 'purchase', 'refund'
    )
),

installs AS (
  SELECT
    resolved_user_id,
    event_date AS cohort_date,
    COALESCE(NULLIF(param_country, ''), NULLIF(geo_country, ''), 'Unknown') AS country,
    LOWER(COALESCE(
      NULLIF(param_platform, ''),
      CASE
        WHEN REGEXP_CONTAINS(event_name, r'_android$') THEN 'android'
        WHEN REGEXP_CONTAINS(event_name, r'_ios$') THEN 'ios'
        ELSE device_os
      END
    )) AS platform,
    LOWER(COALESCE(NULLIF(ts_source, ''), NULLIF(cts_source, ''), NULLIF(utm_source, ''), '')) AS src,
    LOWER(COALESCE(NULLIF(ts_medium, ''), NULLIF(cts_medium, ''), NULLIF(utm_medium, ''), '')) AS medium,
    COALESCE(cts_gclid, '') AS gclid,
    event_timestamp
  FROM events_in_window
  WHERE event_name_base = 'first_open'
),

cohorts AS (
  SELECT
    resolved_user_id,
    ARRAY_AGG(
      STRUCT(cohort_date, country, platform, src, medium, gclid)
      ORDER BY event_timestamp
      LIMIT 1
    )[OFFSET(0)] AS touch
  FROM installs
  GROUP BY resolved_user_id
),

attributed AS (
  SELECT
    resolved_user_id,
    touch.cohort_date AS cohort_date,
    COALESCE(NULLIF(touch.country, ''), 'Unknown') AS country,
    touch.platform AS platform,
```

```
    CASE
      WHEN touch.gclid != ''
        OR touch.medium IN (
          'cpc', 'ppc', 'cpm', 'cpv', 'cpa', 'cpi', 'cpe',
          'paid', 'paidsocial', 'paid-social', 'paid_social',
          'display', 'video', 'shopping', 'uac', 'uai'
        )
        OR touch.medium LIKE '%paid%'
        THEN 'Paid'
      WHEN touch.medium IN ('organic', 'organic-search')
        OR (
          touch.src IN ('google-play', 'googleplay', 'play.google.com', 'itunes', 'appstore', 'app-store', 'apple')
          AND touch.medium NOT LIKE '%paid%'
        )
        THEN 'Organic'
      WHEN touch.src IN ('(direct)', 'direct', '')
        AND touch.medium IN ('(none)', 'none', '', '(direct)')
        THEN 'Direct'
      WHEN touch.src IN ('google-play', 'googleplay', 'itunes', 'appstore', 'apple')
        THEN 'Organic'
      WHEN touch.src != '' OR (touch.medium != '' AND touch.medium NOT IN ('(none)', 'none'))
        THEN 'Organic'
      ELSE 'Direct'
    END AS install_channel
  FROM cohorts
  CROSS JOIN bounds
  WHERE touch.cohort_date BETWEEN bounds.cohort_start AND bounds.cohort_end
),

purchases AS (
  SELECT
    resolved_user_id,
    event_date AS purchase_date,
    CASE
      WHEN event_name_base = 'refund' THEN -ABS(amount_usd)
      ELSE amount_usd
    END AS amount_usd
  FROM events_in_window
  WHERE event_name_base IN ('in_app_purchase', 'purchase', 'refund')
    AND amount_usd IS NOT NULL
),

user_ltv AS (
  SELECT
    a.resolved_user_id,
    a.cohort_date,
    a.country,
    a.platform,
    a.install_channel,
    DATE_DIFF(CURRENT_DATE(), a.cohort_date, DAY) AS cohort_age_days,
    SUM(IF(p.purchase_date BETWEEN a.cohort_date AND DATE_ADD(a.cohort_date, INTERVAL 29 DAY), p.amount_usd, 0)) AS
revenue_30,
    SUM(IF(p.purchase_date BETWEEN a.cohort_date AND DATE_ADD(a.cohort_date, INTERVAL 89 DAY), p.amount_usd, 0)) AS
revenue_90,
    SUM(IF(p.purchase_date BETWEEN a.cohort_date AND DATE_ADD(a.cohort_date, INTERVAL 179 DAY), p.amount_usd, 0)) AS
revenue_180
  FROM attributed a
  LEFT JOIN purchases p
    ON p.resolved_user_id = a.resolved_user_id
   AND p.purchase_date >= a.cohort_date
  GROUP BY
    a.resolved_user_id, a.cohort_date, a.country, a.platform, a.install_channel
)

SELECT
  cohort_date,
  country,
  install_channel,
  COUNT(*) AS installs,
  IF(MAX(cohort_age_days) >= 30, SUM(revenue_30), NULL) AS revenue_30,
  IF(MAX(cohort_age_days) >= 90, SUM(revenue_90), NULL) AS revenue_90,
  IF(MAX(cohort_age_days) >= 180, SUM(revenue_180), NULL) AS revenue_180,
  IF(MAX(cohort_age_days) >= 30, SAFE_DIVIDE(SUM(revenue_30), COUNT(*)), NULL) AS ltv_30,
  IF(MAX(cohort_age_days) >= 90, SAFE_DIVIDE(SUM(revenue_90), COUNT(*)), NULL) AS ltv_90,
  IF(MAX(cohort_age_days) >= 180, SAFE_DIVIDE(SUM(revenue_180), COUNT(*)), NULL) AS ltv_180,
  IF(MAX(cohort_age_days) >= 30, COUNTIF(revenue_30 > 0), NULL) AS payers_30,
  IF(MAX(cohort_age_days) >= 90, COUNTIF(revenue_90 > 0), NULL) AS payers_90,
```

```
  IF(MAX(cohort_age_days) >= 180, COUNTIF(revenue_180 > 0), NULL) AS payers_180,
  IF(MAX(cohort_age_days) >= 30, SAFE_DIVIDE(COUNTIF(revenue_30 > 0), COUNT(*)), NULL) AS paid_rate_30,
  IF(MAX(cohort_age_days) >= 90, SAFE_DIVIDE(COUNTIF(revenue_90 > 0), COUNT(*)), NULL) AS paid_rate_90,
  IF(MAX(cohort_age_days) >= 180, SAFE_DIVIDE(COUNTIF(revenue_180 > 0), COUNT(*)), NULL) AS paid_rate_180
FROM user_ltv
WHERE TRUE
  [[AND country = {{country}}]]
  [[AND platform = {{platform}}]]
  [[AND install_channel = {{install_channel}}]]
GROUP BY cohort_date, country, install_channel
ORDER BY cohort_date, country, install_channel;
```